
SimpleGPT: Improving GPT via A Simple Normalization Strategy

Marco Chen^{*1} Xianbiao Qi^{*†2} Yelin He² Jiaquan Ye² Rong Xiao^{†2}

Abstract

In this work, we revisit Transformer optimization through the lens of second-order geometry and establish a direct connection between architectural design, activation scale, the Hessian matrix, and the maximum tolerable learning rate. We introduce a simple normalization strategy, termed SimpleNorm, which stabilizes intermediate activation scales by construction. Then, by analyzing the Hessian of the loss with respect to network activations, we theoretically show that SimpleNorm significantly reduces the spectral norm of the Hessian, thereby permitting larger stable learning rates. We validate our theoretical findings through extensive experiments on large GPT models at parameter scales 1B, 1.4B, 7B and 8B. Empirically, SimpleGPT, our SimpleNorm-based network, tolerates learning rates $3\times-10\times$ larger than standard convention, consistently demonstrates strong optimization stability, and achieves substantially better performance than well-established baselines. Specifically, when training 7B-scale models for 60K steps, SimpleGPT reduces the training loss from 2.290 to 2.208 compared with Llama2 with QKNorm. Our code is available at <https://github.com/Ocram7/SimpleGPT>.

1. Introduction

Transformer-based large language models (LLMs) (Radford et al., 2018; 2019; Brown et al., 2020; Touvron et al., 2023a;b; Dubey et al., 2024; Chowdhery et al., 2023; Liu et al., 2024; Team, 2023) have achieved state-of-the-art performance across a wide range of tasks. As these models scale in depth and width, optimization stability increasingly constrains performance and scalability. Many architectural components in modern Transformers—such as residual con-

nections (He et al., 2016), normalization layers (Ioffe & Szegedy, 2015; Ba et al., 2016; Zhang & Sennrich, 2019; Large et al., 2024), and nonlinear activations (Shazeer, 2020)—are primarily introduced to stabilize training, and in some cases to increase expressivity. Understanding optimization from a principled perspective is therefore central to the design of scalable Transformer architectures.

Classical optimization theory (Nesterov, 1983; 1998; Nocedal & Wright, 1999; Boyd & Vandenberghe, 2004) provides a precise connection between optimization stability and second-order geometry. For a twice-differentiable objective $\ell(\mathbf{x})$, the local curvature is characterized by the Hessian $\mathbf{H}_{\mathbf{x}\mathbf{x}} = \nabla^2 \ell(\mathbf{x})$. If ℓ is β -smooth, then $\|\nabla \ell(\mathbf{x}) - \nabla \ell(\mathbf{y})\|_2 \leq \beta \|\mathbf{x} - \mathbf{y}\|_2, \forall \mathbf{x}, \mathbf{y}$. Standard results imply that gradient descent is stable only when the **maximum tolerable learning rate** η satisfies

$$\eta \leq \frac{2}{\beta} = \frac{2}{\sup_{\mathbf{x}} \|\mathbf{H}_{\mathbf{x}\mathbf{x}}\|_2},$$

establishing the Hessian spectral norm as the fundamental quantity governing admissible learning rates and convergence behavior.

In contrast, much of the recent literature on Transformer optimization focuses on architectural heuristics without explicitly analyzing their relationship with classical optimization theory. Techniques such as normalization placement (Vaswani et al., 2017; Wang et al., 2019; Henry et al., 2020; Qi et al., 2023a; 2025c;a), residual scaling (He et al., 2016; Bachlechner et al., 2021; Xie et al., 2025), or modified nonlinearities (Hendrycks, 2016; Shazeer, 2020) are typically justified empirically, while their impact on activation scale, Hessian geometry, and thereby optimal learning rates remains implicit. As a result, the theoretical relationship between network design and classical stability conditions is not well understood, despite its relevance to training very deep and large-scale models.

In this work, we bridge this gap by analyzing Transformer architectures through the central lens of Hessian-based optimization theory, while accounting for the role of activation scale. We introduce a simple normalization strategy, termed *SimpleNorm*, which stabilizes intermediate activation scales through normalization *immediately* following linear mappings. Building on this structural property, we analyze the Hessian of the loss with respect to network activations and

^{*}Equal contribution ¹Tsinghua University, marco.ty.chen@gmail.com ²Intellifusion Inc., Correspondence to: Xianbiao Qi <qixianbiao@gmail.com>, Rong Xiao <rongxiao@gmail.com>.

show that SimpleNorm significantly reduces the spectral norm of the Hessian, thereby permitting substantially larger stable learning rates. By grounding Transformer design in classical optimization principles (Nesterov, 1983; 1998; 2013), our framework provides a unified explanation for existing stabilization techniques and offers principled guidance for building scalable and stable models.

Our contributions can be summarized as follows:

- We revisit Transformer optimization through the lens of second-order geometry and establish a direct connection between architectural design, activation scale, the Hessian, and the maximum tolerable learning rate.
- We introduce SimpleGPT, a new GPT architecture based on SimpleNorm, and theoretically show that this design significantly reduces $\|H_{xx}\|_2$, yielding a smaller Lipschitz gradient constant and enabling substantially larger stable learning rates.
- We demonstrate experimentally that these theoretical advantages are accompanied by consistent empirical gains across nanoGPT, Llama2, and Llama3 architectures, for model sizes ranging from 1B to 8B parameters. Specifically, when training 7B-scale models for 60K steps, our method achieves a training loss that is 0.08 lower than that of Llama2 with QKNorm, reducing the loss from 2.290 to 2.208. SimpleGPT also improves held-out language modeling performance and remains competitive on downstream tasks.

Conflict of Interest Disclosure. The authors declare no financial interests beyond the disclosed affiliations.

2. Related Work

Normalization Methods. Normalization has long been a central tool for stabilizing optimization and improving convergence in deep networks. Batch Normalization (BN) (Ioffe & Szegedy, 2015) normalizes activations using mini-batch statistics and has been widely successful in convolutional architectures, but its behavior can depend on batch size and distributed synchronization. Layer Normalization (LN) (Ba et al., 2016) and its variants remove batch dependence by computing statistics across features within each sample and have become the standard in Transformers. Related methods such as Instance Normalization (IN) (Ulyanov et al., 2016), Group Normalization (GN) (Wu & He, 2018), RMSNorm (Zhang & Sennrich, 2019), and nGPT (Loshchilov et al., 2025) further tailor normalization to specific architectural or efficiency constraints.

Normalization Placement in Transformers. Beyond the choice of normalization operator, its *placement* within Transformer blocks plays a critical role in optimization stabil-

ity. The original Transformer architecture adopted post-normalization (PostNorm), in which normalization follows residual addition (Vaswani et al., 2017). Subsequent large-scale practice shifted toward pre-normalization (PreNorm), placing normalization before attention and MLP sublayers to improve trainability in deep networks (Wang et al., 2019).

Recent work further systematizes normalization placement and explores additional insertion points. Deeply Normalized Transformer (DNT) (Qi et al., 2025a) categorizes multiple strategies—including InputNorm, PreNorm, MidNorm, PostNorm, and QKNorm—and motivates them through a Jacobian- and gradient-stability analysis. DNT ultimately combines InputNorm, PreNorm, MidNorm, and QKNorm, while avoiding PostNorm due to its potential training instabilities. Among these placements, QK normalization (QKNorm) (Henry et al., 2020) specifically targets the attention mechanism, stabilizing the geometry of query-key interactions and mitigating softmax saturation.

By treating normalization as a design space over both operator and location, these works emphasize that stability and conditioning can be targeted at specific architectural subcomponents, rather than only at block outputs. As model depth increases, normalization also interacts with residual pathways and initialization. DeepNorm (Wang et al., 2022), for example, modifies residual scaling and initialization to bound parameter updates and control dynamical growth with depth, complementing normalization-placement strategies.

Normalization-Free Transformers. Motivated by the cost/complexity of normalization and the desire for simpler training dynamics, recent work questions whether explicit normalization is necessary in Transformers. *Transformers without Normalization* shows that replacing normalization layers with a simple point-wise nonlinearity, Dynamic Tanh (DyT) (Zhu et al., 2025), can match normalized baselines across tasks, suggesting that an appropriate bounded nonlinearity can provide much of the stability typically attributed to LN/RMSNorm. Building on this, *Stronger Normalization-Free Transformers* (Chen et al., 2025) studies the design of point-wise functions more broadly and reports improved normalization-free performance via a searched function family (e.g., Derf), outperforming LN/RMSNorm/DyT across multiple domains. Despite being framed as normalization-free, these approaches fundamentally operate by controlling the norm of activations through bounded transformations, and can therefore be viewed as a form of implicit normalization.

Positioning of Our Work. While our method can be viewed as a study of normalization placement in Transformers, its key distinction lies in explicitly linking architectural design to second-order optimization geometry. Rather than motivating normalization heuristically or empirically, we analyze how local normalization immediately following linear map-

pings stabilizes activation scale and, in turn, constrains the spectral norm of the Hessian and leads to a smoother optimization landscape. This perspective yields a principled characterization of the maximum tolerable learning rate and provides a unified theoretical explanation for optimization stability in large Transformer models.

3. Preliminaries

We consider the unconstrained convex optimization problem $\min_{\mathbf{x} \in \mathbb{R}^d} f(\mathbf{x})$, where $f: \mathbb{R}^d \rightarrow \mathbb{R}$ is *differentiable*.

3.1. Convex and Smoothed Optimization

Lipschitz Gradient Smoothness. If f is twice differentiable, its second-order Taylor expansion at point $\mathbf{x}$ is

$$f(\mathbf{y}) \approx f(\mathbf{x}) + \langle \nabla f(\mathbf{x}), \mathbf{y} - \mathbf{x} \rangle + \frac{1}{2}(\mathbf{y} - \mathbf{x})^\top \nabla^2 f(\mathbf{x})(\mathbf{y} - \mathbf{x}).$$

The second-order term captures the local curvature.

Definition 3.1 (β -smoothness). The function f is said to be β -smooth if

$$\|\nabla f(\mathbf{y}) - \nabla f(\mathbf{x})\|_2 \leq \beta \|\mathbf{y} - \mathbf{x}\|_2, \forall \mathbf{y}, \mathbf{x}.$$

For convex and differentiable functions, β -smoothness is equivalent to the following quadratic upper bound:

$$f(\mathbf{y}) \leq f(\mathbf{x}) + \langle \nabla f(\mathbf{x}), \mathbf{y} - \mathbf{x} \rangle + \frac{\beta}{2} \|\mathbf{y} - \mathbf{x}\|_2^2, \forall \mathbf{y}, \mathbf{x}.$$

This inequality plays a central role in step-size (learning rate) selection for optimization methods.

Gradient Descent and Learning Rate. Consider the standard gradient descent iteration

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \eta \nabla f(\mathbf{x}_k),$$

where $\eta > 0$ is the learning rate. We wish to understand how the choice of η depends on the smoothness constant β , and how this choice affects convergence.

Descent Condition. Evaluating the quadratic upper bound with $\mathbf{y} = \mathbf{x}_{k+1} = \mathbf{x}_k - \eta \nabla f(\mathbf{x}_k)$ and $\mathbf{x} = \mathbf{x}_k$, we obtain

$$f(\mathbf{x}_{k+1}) \leq f(\mathbf{x}_k) - \eta \|\nabla f(\mathbf{x}_k)\|_2^2 + \frac{\beta \eta^2}{2} \|\nabla f(\mathbf{x}_k)\|_2^2.$$

Rearranging terms gives

$$f(\mathbf{x}_{k+1}) \leq f(\mathbf{x}_k) - \left(\eta - \frac{\beta \eta^2}{2} \right) \|\nabla f(\mathbf{x}_k)\|_2^2.$$

A sufficient condition for monotone decrease of the objective is therefore

$$0 < \eta \leq \frac{2}{\beta}. \quad (1)$$

This bound characterizes the *stability region* of gradient descent for convex β -smooth functions. $\frac{2}{\beta}$ is usually known as the maximum tolerable learning rate.

For convex β -smooth problems, among all fixed learning rates that ensure descent, the canonical choice is typically $\eta = \frac{1}{\beta}$. This choice balances progress and stability and leads to the sharpest worst-case guarantees.

3.2. Gradient and Hessian of a Linear Projection

Given linear projection $\mathbf{y} = \mathbf{W}\mathbf{x}$ and loss function ℓ , the gradient and Hessian of ℓ with respect to $\mathbf{y}$ are,

$$\mathbf{g}_y := \frac{\partial \ell}{\partial \mathbf{y}}^\top, \quad \mathbf{H}_{yy} := \frac{\partial^2 \ell}{\partial \mathbf{y} \partial \mathbf{y}^\top}.$$

The Jacobian of $\mathbf{y}$ with respect to $\mathbf{x}$ is $\mathbf{J}_x^\mathbf{y} = \frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \mathbf{W}$. Hence, according to the chain rule, we have

$$\mathbf{g}_x = \frac{\partial \ell}{\partial \mathbf{x}}^\top = \mathbf{J}_x^\mathbf{y} \mathbf{g}_y = \mathbf{W}^\top \mathbf{g}_y,$$

and the Hessian matrix with respect to $\mathbf{x}$ is

$$\mathbf{H}_{xx} = \frac{\partial^2 \ell}{\partial \mathbf{x} \partial \mathbf{x}^\top} = \mathbf{J}_x^\mathbf{y} \mathbf{H}_{yy} \mathbf{J}_x^\mathbf{y} = \mathbf{W}^\top \mathbf{H}_{yy} \mathbf{W}. \quad (2)$$

4. Methodology

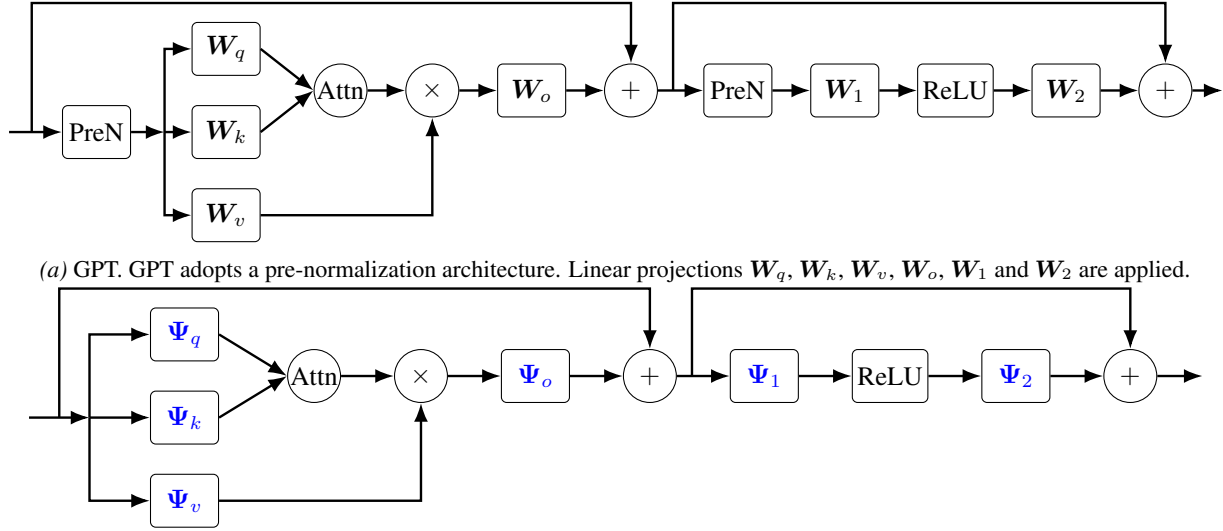
4.1. SimpleNorm: A Unified Normalization Strategy

Definition of SimpleNorm. We define *SimpleNorm* as placing a normalization operator *immediately* after a linear mapping. Given an input vector $\mathbf{x} \in \mathbb{R}^m$ and a linear transformation $\mathbf{W} \in \mathbb{R}^{d \times m}$, we abstract SimpleNorm as a primitive operator

$$\Psi(\mathbf{x}) = \text{Norm}(\mathbf{W}\mathbf{x}), \quad (3)$$

where $\text{Norm}(\cdot)$ is a normalization operator such as LayerNorm or RMSNorm. SimpleNorm is motivated by a simple yet effective *placement* strategy, rather than algebraic complexity. In contrast to existing normalization techniques that typically operate at the level of residual blocks, hidden states, or parameter reparameterization, SimpleNorm enforces normalization *locally and immediately* after linear mapping, treating “linear mapping immediately followed by a normalization” as a single, unified operator.

Definition of SimpleGPT. As illustrated in Figure 1, *SimpleGPT* uses SimpleNorm as a fundamental building block. SimpleNorm is systematically inserted wherever a linear



(a) GPT. GPT adopts a pre-normalization architecture. Linear projections W_q, W_k, W_v, W_o, W_1 and W_2 are applied.

(b) SimpleGPT. SimpleGPT replaces all linear layers with **SimpleNorm** operator, denoted by Ψ . In SimpleGPT, we do not use prenorm.

Figure 1. SimpleGPT vs. GPT. This figure compares the standard GPT block with the proposed SimpleGPT block, highlighting the structural simplifications introduced by SimpleNorm.

layer appears, including MLP projections, attention projections (Q, K, V), output projections, and gating or memory-related modules. Take Figure 1 as an example, normalization is inserted after the W_q, W_k, W_v, W_o, W_1 , and W_2 projections. In architectures that employ SwiGLU (Shazeer, 2020) instead of MLP, SimpleGPT inserts normalization after $W_q, W_k, W_v, W_o, W_1, W_2$, and W_3 .

Instantiating SimpleNorm with RMSNorm In this work, we instantiate $\text{Norm}(\cdot)$ with RMSNorm (Zhang & Sennrich, 2019). Hence, SimpleNorm is now defined as:

$$\Psi(x; W, \gamma) = \gamma \odot \sqrt{d} \frac{Wx}{\|Wx\|_2}, \quad (4)$$

where $x \in \mathbb{R}^m$, $W \in \mathbb{R}^{d \times m}$, and W, γ are learnable parameters, and $\odot$ denotes element-wise multiply.

For later analysis, we define the intermediate variables

$$z = Wx, \quad s = \|z\|_2, \quad u = \frac{z}{s}, \\ P = I - uu^\top, \quad D = \text{Diag}(\gamma),$$

so that $\Psi(x; W, \gamma) = \sqrt{d} Du$.

Core Properties of SimpleNorm The following sections prove two important mechanisms of SimpleNorm: SimpleNorm directly stabilizes the scale of activations to be on the order of $\sqrt{d}$, and SimpleNorm constrains the spectral norm of the Hessian of the loss w.r.t. the activations, smoothing the loss landscape and enabling larger learning rates. Moreover, we present a hypothesis for why, in ad-

dition to the predicted optimization stability, SimpleNorm also exhibits strong empirical performance.

4.2. Mechanism I: Stable Activation Scale

By construction, SimpleNorm stabilizes the scale of intermediate activations by normalizing *immediately* after each linear mapping. Recall that

$$\Psi(x; W, \gamma) = \sqrt{d} Du \quad \text{with } \|u\|_2 = 1.$$

Then

$$\|\Psi(x; W, \gamma)\|_2 = \sqrt{d} \|Du\|_2.$$

In particular, letting $\gamma_{\min} = \min_i |\gamma_i|$ and $\gamma_{\max} = \max_i |\gamma_i|$, we have the bound

$$\gamma_{\min} \sqrt{d} \leq \|\Psi(x)\|_2 \leq \gamma_{\max} \sqrt{d}.$$

Thus, up to the learned per-dimension scaling γ , each projection is rescaled to have norm on the order of $\sqrt{d}$. Consequently, SimpleNorm prevents intermediate representation norms from drifting with depth or weight growth, eliminating a common source of activation explosion. Mechanism II shows how, in addition to activations, SimpleNorm also stabilizes curvature via the gradient Lipschitz constant.

4.3. Mechanism II: Smoother Loss Landscape

Smoothness and the Hessian. The smoothness of the objective directly constrains optimization stability: larger curvature implies smaller safe learning rates. Given a twice-differentiable β -smooth objective $\ell(x)$, we quantify local curvature by the activation Hessian $H_{xx} = \nabla_{xx}^2 \ell$. Specifically, the supremum of the spectral norm of the Hessian

upper bounds local curvature and governs gradient stability:

$$\beta = \sup_x \|\mathbf{H}_{xx}(\mathbf{x})\|_2.$$

To show SimpleNorm yields a smoother landscape, we prove two results: (i) the SimpleNorm Hessian decomposes as $\mathbf{H}_{xx} = \mathbf{L} + \mathbf{C}$ and in high dimension $\|\mathbf{C}\|_2 \ll \|\mathbf{L}\|_2$; (ii) compared to a linear projection whose curvature scales as $\|\mathbf{W}\|_2^2$, the SimpleNorm curvature is scale-invariant with respect to $\|\mathbf{W}\|_2$. Combined, these imply $\|\mathbf{H}_{xx}^{\text{sn}}\|_2 \ll \|\mathbf{H}_{xx}^{\text{lin}}\|_2$ since $\|\mathbf{W}\|_2$ generally grows during training.

SimpleNorm Derivatives. First, we compute the first-order gradient and second-order Hessian of ℓ with respect to $\mathbf{x}$.

Given $\mathbf{y} = \sqrt{d} \mathbf{D} \mathbf{u}$, let

$$\ell = \ell(\mathbf{y}), \quad \mathbf{g}_y := \nabla_y \ell, \quad \mathbf{H}_{yy} := \nabla_{yy}^2 \ell.$$

First-order derivative. The Jacobian of the normalization satisfies $\frac{\partial \mathbf{u}}{\partial \mathbf{z}} = \frac{1}{s} \mathbf{P}$, which yields the Jacobian of $\mathbf{y}$ with respect to $\mathbf{x}$:

$$\mathbf{J}_x^y := \frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \frac{\sqrt{d}}{s} \mathbf{D} \mathbf{P} \mathbf{W}.$$

Applying the chain rule, the gradient is

$$\nabla_x \ell = \mathbf{J}_x^{y\top} \mathbf{g}_y = \frac{\sqrt{d}}{s} \mathbf{W}^\top \mathbf{P} \mathbf{D} \mathbf{g}_y. \quad (5)$$

Second-order derivative. Differentiating the gradient leads to the standard decomposition

$$\mathbf{H}_{xx} = \nabla_x^2 \ell = \underbrace{\mathbf{J}_x^{y\top} \mathbf{H}_{yy} \mathbf{J}_x^y}_{\text{Gauss-Newton term}} + \underbrace{\mathbf{C}}_{\text{curvature term}}, \quad (6)$$

where the first term is the Gauss–Newton component

$$\mathbf{J}_x^{y\top} \mathbf{H}_{yy} \mathbf{J}_x^y = \frac{d}{s^2} \mathbf{W}^\top \mathbf{P} \mathbf{D} \mathbf{H}_{yy} \mathbf{D} \mathbf{P} \mathbf{W}, \quad (7)$$

and the second term is from the curvature of the normalization,

$$\mathbf{C} = -\frac{\sqrt{d}}{s^2} \mathbf{W}^\top \left(\mathbf{P} \mathbf{D} \mathbf{g}_y \mathbf{u}^\top + \mathbf{u}^\top \mathbf{D} \mathbf{g}_y \mathbf{P} + \mathbf{u} \mathbf{g}_y^\top \mathbf{D} \mathbf{P} \right) \mathbf{W}. \quad (8)$$

Please see [Appendix A](#) for a detailed derivation of $\nabla_x \ell$ and $\nabla_x^2 \ell$ for SimpleNorm. For completeness, derivations of $\nabla_\gamma \ell$, $\nabla_\gamma^2 \ell$, $\nabla_{\mathbf{W}} \ell$ and $\nabla_{\text{vec}(\mathbf{W})}^2 \ell$ with $\mathbf{y} = \gamma \odot \sqrt{d} \frac{\mathbf{W} \mathbf{x}}{\|\mathbf{W} \mathbf{x}\|_2}$ and $\nabla_{\mathbf{W}} \ell$ and $\nabla_{\text{vec}(\mathbf{W})}^2 \ell$ with $\mathbf{y} = \mathbf{W} \mathbf{x}$ are also provided in [Appendix C](#) and [Appendix B](#)

Gauss–Newton Term Dominates in High Dimension.

Next, we show that under standard high-dimensional and non-pathological conditions, the Gauss–Newton term dominates the curvature induced by normalization.

Theorem 4.1 (Gauss–Newton dominance for SimpleNorm). *Let $\mathbf{H}_{xx} = \nabla_x^2 \ell$ denote the activation Hessian induced by SimpleNorm for a twice-differentiable objective $\ell(\mathbf{y})$. Then, the Hessian decomposes as*

$$\mathbf{H}_{xx} = \mathbf{L} + \mathbf{C}, \quad \mathbf{L} = (\mathbf{J}_x^y)^\top \mathbf{H}_{yy} \mathbf{J}_x^y,$$

where $\mathbf{C}$ is the curvature term induced by normalization.

Assume $\|\mathbf{x}\|_2 = \sqrt{d}$, $\mathbf{D} = \mathbf{I}$, $\mathbf{W} \in \mathbb{R}^{d \times d}$ has high effective rank $\|\mathbf{W}\|_F^2 / \|\mathbf{W}\|_2^2 \geq cd$, and the input and loss derivatives are not pathologically aligned with $\mathbf{W}$. Define

$$\kappa := \left\| \frac{\sqrt{d} \mathbf{W}}{\|\mathbf{W} \mathbf{x}\|_2} \right\|_2.$$

Then $\kappa = \Theta(1)$ with high probability, and there exists a constant $\tau = \Theta(1)$ such that

$$\|\mathbf{L}\|_2 = \tau \kappa^2 \|\mathbf{H}_{yy}\|_2, \quad \|\mathbf{C}\|_2 \leq \frac{3\kappa^2}{\sqrt{d}} \|\mathbf{g}_y\|_2.$$

In particular, if $\|\mathbf{g}_y\|_2 = O(\|\mathbf{H}_{yy}\|_2)$, then

$$\|\mathbf{C}\|_2 \ll \|\mathbf{L}\|_2$$

so the Gauss–Newton term dominates the Hessian w.h.p.

A complete proof is given in [Appendix D](#). Although [Theorem 4.1](#) is stated under the simplifying assumption $\mathbf{D} = \mathbf{I}$, we empirically verify that the learned RMSNorm gain parameters remain in a moderate range during training; see [Appendix F](#) for more details.

SimpleNorm Hessian is Weight Scale-Invariant. Finally, we compare the SimpleNorm Hessian’s magnitude to that of a plain linear projection. We show that linear curvature grows quadratically with the weight matrix spectral norm $\|\mathbf{W}\|_2$, whereas SimpleNorm removes this dependence.

Theorem 4.2 (Linear curvature scales with $\|\mathbf{W}\|_2^2$ while SimpleNorm does not). *Let $\ell = \ell(\mathbf{y})$ be twice differentiable, with $\mathbf{H}_{yy} = \nabla_{yy}^2 \ell$ and $\mathbf{g}_y = \nabla_y \ell$.*

Consider the linear mapping with its Hessian

$$\mathbf{y}_1 = \mathbf{W}_1 \mathbf{x}, \quad \mathbf{H}_{xx}^{\text{lin}} = \mathbf{W}_1^\top \mathbf{H}_{yy} \mathbf{W}_1,$$

and the SimpleNorm mapping with its Hessian

$$\mathbf{y}_2 = \mathbf{D} \frac{\sqrt{d} \mathbf{W}_2 \mathbf{x}}{\|\mathbf{W}_2 \mathbf{x}\|_2}, \quad \mathbf{H}_{xx}^{\text{sn}} = \mathbf{L} + \mathbf{C}.$$

Assume $\mathbf{H}_{y_1 y_1} = \mathbf{H}_{y_2 y_2} := \mathbf{H}_{yy}$, $\mathbf{W}_1 = \mathbf{W}_2 := \mathbf{W}$, and that the conditions of [Theorem 4.1](#) hold, such that $\|\mathbf{L}\|_2 \gg \|\mathbf{C}\|_2$. Then, with high-probability,

$$\|\mathbf{H}_{xx}^{\text{sn}}\|_2 = \Theta(\kappa^2 \|\mathbf{H}_{yy}\|_2), \quad \kappa^2 = \frac{d}{\|\mathbf{W} \mathbf{x}\|_2^2} = \Theta(1),$$

where $\widetilde{\mathbf{W}} = \mathbf{W} / \|\mathbf{W}\|_2$.

Moreover, if the range of $\widetilde{\mathbf{W}}$ is not adversarially aligned with the leading eigenspace of $\mathbf{H}_{yy}$, then there exists a constant $c_{\text{lin}} = \Theta(1)$ such that

$$\|\mathbf{H}_{xx}^{\text{lin}}\|_2 = \|\mathbf{W}^\top \mathbf{H}_{yy} \mathbf{W}\|_2 \geq c_{\text{lin}} \|\mathbf{W}\|_2^2 \|\mathbf{H}_{yy}\|_2.$$

Consequently, as $\|\mathbf{W}\|_2$ grows during training,

$$\|\mathbf{H}_{xx}^{\text{lin}}\|_2 \gg \|\mathbf{H}_{xx}^{\text{sn}}\|_2 \quad (\text{with high probability}).$$

Intuitively, SimpleNorm removes the dependence of curvature on weight scale by normalizing activations, whereas a linear projection amplifies curvature as $\|\mathbf{W}\|_2$ grows. We provide a proof for [Theorem 4.2](#) in [Appendix E](#).

SimpleNorm Enables Larger Learning Rates. For a twice-differentiable β -smooth objective, the maximum stable learning rate of gradient descent is inversely proportional to β , the Lipschitz constant of the gradient, which is equivalent to the supremum of the spectral norm of the Hessian: $\eta \leq \frac{2}{\beta}$ where $\beta = \sup_{\mathbf{x}} \|\mathbf{H}_{xx}(\mathbf{x})\|_2$

[Theorem 4.1](#) and [Theorem 4.2](#) establish that, under standard high-dimensional and non-pathological conditions, the SimpleNorm Hessian is invariant to the spectral norm of the weight matrix whereas the Hessian of a linear projection scales quadratically with the weight norm.

Consequently, since the weight spectral norm generally grows throughout training, the SimpleNorm-based SimpleGPT architecture has a smoother loss landscape that can tolerate significantly larger learning rates compared to methods based on direct linear projections.

4.4. Interpretation: Beyond Optimization Stability

We have established two core properties of SimpleNorm: (i) it stabilizes activation scale at $\Theta(\sqrt{d})$, and (ii) it smooths the loss landscape by constraining the spectral norm of the activation Hessian, enabling larger and more stable learning rates. Although these properties explain the improved optimization stability of SimpleNorm, they do not fully account for the strong empirical performance observed in [Section 5](#).

We hypothesize that SimpleNorm provides additional benefits at a more *global* representational level. By normalizing immediately after each linear projection, SimpleNorm ensures that every layer induces a genuinely nonlinear transformation, even in regimes where the surrounding network would otherwise behave nearly linearly. This effectively increases the depth of nonlinear interactions and enhances expressive capacity without increasing parameter count.

Under this view, SimpleNorm improves performance through a dual effect: locally, by improving optimization

geometry via reduced curvature variability; and globally, by increasing expressiveness through pervasive normalization-induced nonlinearity. We believe this combination explains why SimpleNorm yields consistent empirical gains beyond what would be expected from learning-rate stability alone.

4.5. Use `torch.compile` to Speedup Training

Normalization layers are memory-bound and frequently executed, making them a potential bottleneck. By fusing reduction and pointwise operations and leveraging `torch.compile`, SimpleNorm’s increased normalization overhead is largely amortized, resulting in around a 3% training-time increase compared to GPT with QKNorm.

5. Experiments

Experimental Settings. We evaluate SimpleNorm on three Transformer backbones: nanoGPT, Llama2, and Llama3. SimpleNorm is applied to all Transformer blocks, excluding the embedding and output layers. All models are trained using the AdamW optimizer (Kingma & Ba, 2014; Loshchilov & Hutter, 2019) with cosine learning-rate scheduling with bfloat16 precision. Learning rates are tuned for each method. Since SimpleNorm permits significantly larger stable learning rates, we adjust weight decay accordingly. Additional architectural, hyperparameter, and training details are provided in [Appendix G](#) and [Appendix H](#).

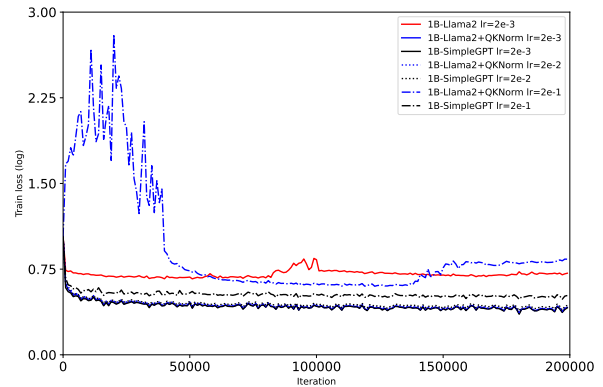


Figure 2. The largest admissible learning rate for Llama2-1B, Llama2-1B with QKNorm, and SimpleGPT-1B.

5.1. Largest Tolerable Learning Rate

We evaluate the largest tolerable learning rate by comparing optimization stability across different normalization schemes while keeping all other training settings fixed. As shown in [Figure 2](#), PreNorm already exhibits convergence issues at a learning rate of 2×10^{-3} . In contrast, PreNorm+QKNorm remains stable at 2×10^{-3} and 2×10^{-2} , but becomes unstable when the learning rate is increased to 2×10^{-1} . SimpleNorm shows stable convergence at both

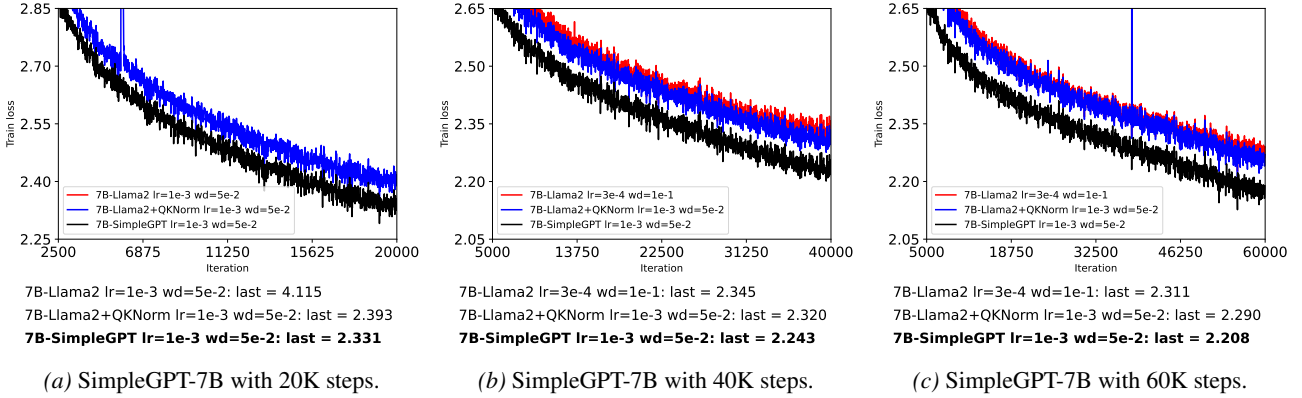


Figure 3. The training loss curves of Llama2-7B, Llama2-7B with QKNorm and SimpleGPT-7B under 20K, 40K and 60K training steps.

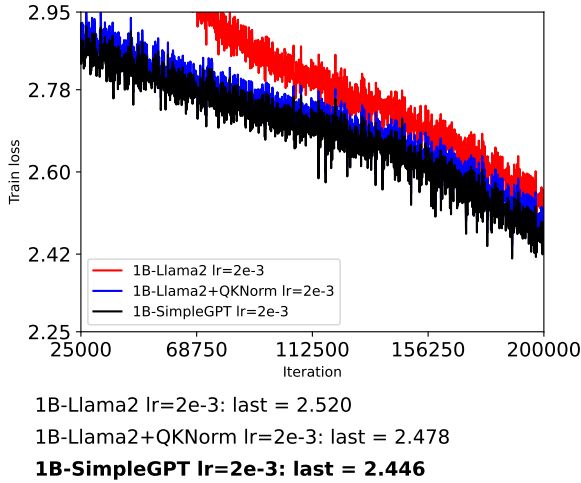


Figure 4. The training loss curves of Llama2-1B, Llama2-1B with QKNorm and SimpleGPT-1B under 200K training steps.

2×10^{-3} and 2×10^{-2} , and is notably more stable than PreNorm+QKNorm at 2×10^{-1} . Overall, these results suggest that SimpleNorm consistently tolerates larger learning rates, indicating improved optimization robustness.

5.2. SimpleGPT-1B Based on Llama2

In this subsection, we evaluate SimpleGPT-1B and compare it against the standard Llama2-1B as well as Llama2-1B with QKNorm. We train the model for 200K steps (following (Zhang et al., 2024)), with a global batch size of 256 and a sequence length of 512, resulting in approximately 26B training tokens. We train all models on the C4 dataset following the same training recipe as their corresponding baselines. The results are presented in Figure 4. The loss curve is smoothed by 80% in Tensorboard. For all experiments, we report the training loss of the last step.

In Figure 4, SimpleGPT-1B achieves notable improvement over Llama2-1B with QKNorm. Specifically, the training

loss is reduced from 2.478 to 2.446, corresponding to an absolute improvement of 0.032. Hence, SimpleGPT provides measurable gains, even at the small 1B scale.

5.3. SimpleGPT-7B Based on Llama2

We compare SimpleGPT-7B against the standard Llama2-7B and Llama2-7B with QKNorm in Figure 3. We train the models for 20K, 40K, and 60K steps, corresponding to approximately 8B, 16B, and 24B tokens, respectively. All models are trained on the C4 dataset following the same training recipe as their corresponding baselines. SimpleGPT-7B uses a 0.001 learning rate, which is $3\times$ larger than that used in the Llama2-7B (Touvron et al., 2023b) model.

We make the following observations. First, the performance gain of SimpleGPT over Llama2+QKNorm is consistently significant throughout training: the improvement reaches 0.062 at 20K steps, increases to 0.077 at 40K steps, and remains at a comparable level (0.082) at 60K steps. Second, SimpleGPT maintains more stable training dynamics compared to Llama2 with QKNorm. Third, as training progresses and more tokens are observed, the relative improvement does not diminish, indicating that the advantage of SimpleGPT is stable rather than a transient early-training effect. Finally, we observe a clear scaling trend with respect to model size. While the 1B model trained on 26B tokens achieves a modest improvement of approximately 0.03, the 7B model trained on 24B tokens exhibits a substantially larger gain of 0.08.

5.4. SimpleGPT-8B Based on Llama3

At the 8B scale, our experiments are based on the Llama3-8B architecture. We train both SimpleGPT-8B and Llama3-8B on the C4 dataset with a global batch size of 192 and a sequence length of 2048. We conduct training for 20K steps, corresponding to approximately 8B training tokens. We do not train for more steps due to compute constraints.

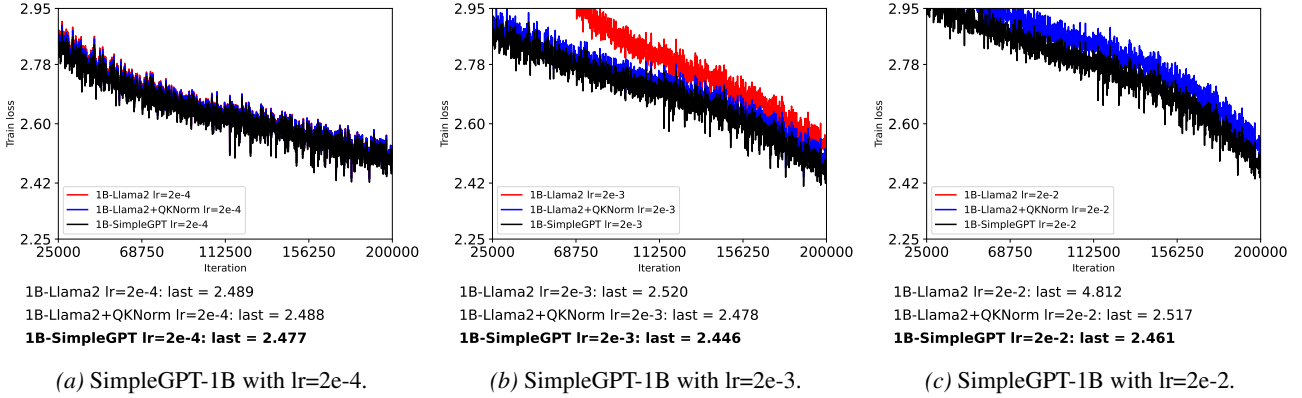


Figure 5. Overall comparison across Llama2-1B, Llama2-1B with QKNorm and SimpleGPT-1B under three different learning rates. Adam-mini uses a 2×10^{-4} learning rate. In SimpleGPT, we enable a $10\times$ learning rate and obtain better performance.

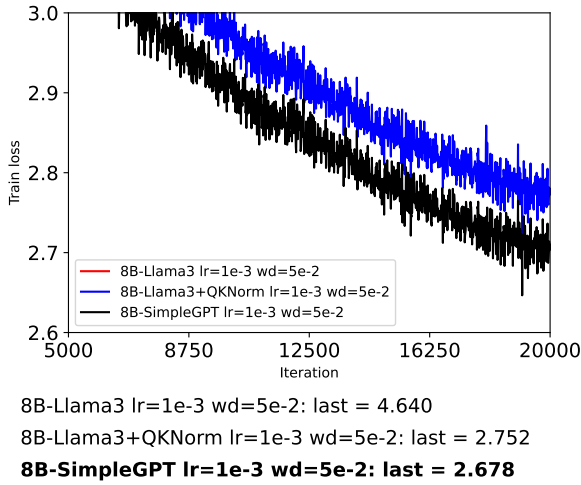


Figure 6. The training loss curves of Llama3-8B, Llama3-8B with QKNorm and SimpleGPT-8B.

SimpleGPT-8B employs a $3\times$ larger learning rate than Llama3-8B, and as shown in Figure 6, achieves a substantially lower training loss. Moreover, the magnitude of the performance gain is consistent with that observed for the 7B model, suggesting that our method exhibits favorable scaling behavior with increasing model size.

5.5. SimpleGPT-1.4B Based on nanoGPT

Finally, we evaluate SimpleGPT-1.4B on the nanoGPT code base. All models are trained for 100K steps, corresponding to approximately 50B tokens. SimpleGPT-1.4B is trained using a learning rate that is $3\times$ larger than the baseline.

We report validation losses in Figure 7. Note that, since validation loss is recorded once every 1,000 steps, the curves in Figure 7 appear different compared to earlier figures.

We observe that GPT-2 with QKNorm achieves nearly identical performance to the original GPT-2, indicating that

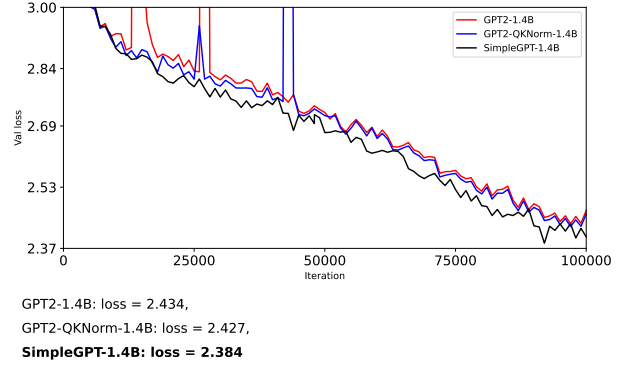


Figure 7. The validation loss curves of GPT2 1.4B, GPT2 1.4B with QKNorm and SimpleGPT 1.4B under 100K training steps.

QKNorm alone provides limited benefits in this setting. Consistent with the results on Llama2-1B, SimpleGPT-1.4B Based on nanoGPT yields an improvement of approximately 0.043. These findings suggest that the gains introduced by SimpleNorm are stable across architectures.

5.6. Held-Out and Downstream Evaluation

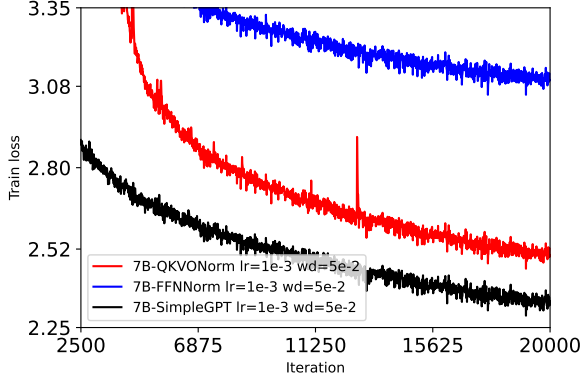
While the preceding experiments primarily evaluate optimization behavior through training and validation losses, we further examine whether the gains of SimpleGPT translate to held-out and downstream performance.

We compare SimpleGPT-7B with Llama2-7B and Llama2-7B with QKNorm. For held-out language modeling evaluation, we report bits-per-byte (bpb) on C4 validation and Wiki, where lower is better. For downstream evaluation, we report accuracy on WinoGrande and normalized accuracy on PIQA and HellaSwag, where higher is better.

As shown in Table 1, SimpleGPT-7B achieves lower bpb on both C4 validation and Wiki, indicating improved held-out language modeling performance. On downstream tasks, SimpleGPT-7B performs competitively with Llama2-7B

Table 1. Held-out and downstream evaluation of SimpleGPT-7B, Llama2-7B with QKNorm, and Llama2-7B. Lower is better for bpb, and higher is better for accuracy-based metrics.

Dataset / Metric	SimpleGPT-7B	Llama2-7B + QKNorm	Llama2-7B
C4 val bpb ↓	0.7999	0.8327	0.8371
Wiki bpb ↓	0.7896	0.8188	0.8227
WinoGrande Acc. ↑	53.99	52.88	49.65
PIQA Acc.-norm ↑	65.61	65.77	64.03
HellaSwag Acc.-norm ↑	45.92	45.33	43.40



7B-QKVNorm lr=1e-3 wd=5e-2: last = 2.494

7B-FFNNorm lr=1e-3 wd=5e-2: last = 3.088

7B-SimpleGPT lr=1e-3 wd=5e-2: last = 2.331

Figure 8. Ablation on where SimpleNorm is applied in Llama2-7B. Comparison between full SimpleGPT, applying SimpleNorm only to the attention projections ($\mathbf{W}_q, \mathbf{W}_k, \mathbf{W}_v, \mathbf{W}_o$), and applying SimpleNorm only to the SwiGLU projections ($\mathbf{W}_1, \mathbf{W}_2, \mathbf{W}_3$).

with QKNorm, achieving slightly higher scores on WinoGrande and HellaSwag while remaining comparable on PIQA. These results suggest that the optimization improvements introduced by SimpleNorm are not merely reflected in training loss, but also lead to improved held-out performance and competitive downstream transfer.

5.7. Ablation Study and Additional Comparisons

Learning-Rate Sensitivity. As shown in Figure 5, we compare Llama2-1B, Llama2-1B with QKNorm, and SimpleGPT-1B under three learning-rate settings to assess whether the improvement of SimpleGPT is solely due to using a larger learning rate.

At the small learning rate 2×10^{-4} , all methods train stably, but SimpleGPT-1B achieves the lowest loss. As the learning rate increases to 2×10^{-3} and 2×10^{-2} , the original Llama2-1B increasingly degrades, while QKNorm remains trainable and SimpleGPT consistently achieves the strongest loss. Overall, these results indicate that SimpleGPT provides gains even under matched learning rates and improves robustness to larger learning rates.

Ablation on SimpleNorm Placement. We further ablate where SimpleNorm should be applied within the Trans-

former block. Using Llama2-7B as the base architecture, we compare the full SimpleGPT design with two partial variants: one that applies SimpleNorm only to the attention projections ($\mathbf{W}_q, \mathbf{W}_k, \mathbf{W}_v, \mathbf{W}_o$), and one that applies SimpleNorm only to the SwiGLU projections ($\mathbf{W}_1, \mathbf{W}_2, \mathbf{W}_3$).

As shown in Figure 8, the full SimpleGPT architecture is more stable and achieves significantly lower training loss than both partial variants. This suggests that the benefit of SimpleNorm does not come from a single isolated insertion point, but from systematically normalizing the outputs of all linear mappings across the block.

We also compare SimpleGPT with SandwichNorm (Ding et al., 2021) in Appendix I. SimpleGPT achieves lower training loss, further suggesting the importance of normalization placement immediately after linear mappings.

5.8. Practical Overhead of SimpleGPT

Training Time. We compare the training speed of our SimpleGPT-8B model with that of Llama3-8B with QKNorm. On average, Llama3-8B requires 1553 ms per training step while SimpleGPT-8B takes 1603 ms per step. This corresponds to a slowdown of around 3%, which can likely be further reduced through kernel-level fusion or more optimized normalization kernels.

Memory Overhead. We also measure the training memory footprint under the Llama2-7B setting on 8 A800 GPUs, with global batch configuration $8 \times 6 \times 4 \times 2048$ (GPUs, gradient accumulation steps, local batch size, and sequence length). The peak memory usage is 54.4 GB for Llama2-7B with PreNorm, 58.5 GB for Llama2-7B with PreNorm+QKNorm, and 60.5 GB for SimpleGPT-7B. Thus, SimpleGPT adds approximately 2.0 GB per GPU over QKNorm in this setting.

6. Conclusion

In this work, we revisit Transformer optimization from a second-order perspective and establish a direct connection between architectural design, the Hessian matrix, and optimization stability. By introducing SimpleNorm and analyzing its induced Hessian structure, we show that reducing the Hessian norm of the activation with respect to the loss enables substantially larger admissible learning rates. The resulting model, **SimpleGPT**, reliably supports learning rates up to $3 \times 10 \times$ larger than strong baselines while maintaining stable optimization. Across extensive experiments on nanoGPT, Llama2, and Llama3 style models, spanning parameter scales from 1B to 8B, our method consistently achieves substantially stronger performance than GPT with QKNorm. Importantly, these gains are obtained with modest additional training-time and memory overhead.

Acknowledgments

This work was supported in part by the National Natural Science Foundation of China under Grant No. 62576048.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Ba, J. L., Kiros, J. R., and Hinton, G. E. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- Bachlechner, T., Majumder, B. P., Mao, H., Cottrell, G., and McAuley, J. Rezero is all you need: Fast convergence at large depth. In *Uncertainty in Artificial Intelligence*, pp. 1352–1361. PMLR, 2021.
- Boyd, S. and Vandenberghe, L. *Convex optimization*. Cambridge university press, 2004.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33: 1877–1901, 2020.
- Chen, M., Lu, T., Zhu, J., Sun, M., and Liu, Z. Stronger normalization-free transformers. *arXiv preprint arXiv:2512.10938*, 2025.
- Chowdhery, A., Narang, S., Devlin, J., Bosma, M., Mishra, G., Roberts, A., Barham, P., Chung, H. W., Sutton, C., Gehrmann, S., et al. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24(240):1–113, 2023.
- Ding, M., Yang, Z., Hong, W., Zheng, W., Zhou, C., Yin, D., Lin, J., Zou, X., Shao, Z., Yang, H., et al. Cogview: Mastering text-to-image generation via transformers. *Advances in neural information processing systems*, 34: 19822–19835, 2021.
- Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Yang, A., Fan, A., et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Gage, P. A new algorithm for data compression. *The C Users Journal*, 12(2):23–38, 1994.
- Golub, G. H. and Van Loan, C. F. *Matrix Computations - 4th Edition*. Johns Hopkins University Press, 2013.
- He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016.
- Hendrycks, D. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415*, 2016.
- Henry, A., Dachapally, P. R., Pawar, S. S., and Chen, Y. Query-key normalization for transformers. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 4246–4253, 2020.
- Horn, R. A. and Johnson, C. R. *Matrix analysis*. Cambridge university press, 2012.
- Ioffe, S. and Szegedy, C. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*, pp. 448–456. PMLR, 2015.
- Karpathy, A. NanoGPT. <https://github.com/karpathy/nanoGPT>, 2022.
- Kingma, D. P. and Ba, J. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- Large, T., Liu, Y., Huh, M., Bahng, H., Isola, P., and Bernstein, J. Scalable optimization in the modular norm. *arXiv preprint arXiv:2405.14813*, 2024.
- Liu, A., Feng, B., Xue, B., Wang, B., Wu, B., Lu, C., Zhao, C., Deng, C., Zhang, C., Ruan, C., et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- Loshchilov, I. and Hutter, F. Fixing weight decay regularization in adam. In *International Conference on Learning Representations*, 2019.
- Loshchilov, I., Hsieh, C.-P., Sun, S., and Ginsburg, B. ngpt: Normalized transformer with representation learning on the hypersphere. *The Thirteenth International Conference on Learning Representations*, 2025.
- Magnus, J. R. and Neudecker, H. *Matrix differential calculus with applications in statistics and econometrics*. John Wiley & Sons, 2019.
- Nesterov, Y. A method for unconstrained convex minimization problem with the rate of convergence $O(1/k^2)$. In *Doklady an ussr*, volume 269, pp. 543–547, 1983.
- Nesterov, Y. Introductory lectures on convex programming, volume i: Basic course, July 1998. Lecture notes.
- Nesterov, Y. *Introductory lectures on convex optimization: A basic course*, volume 87. Springer Science & Business Media, 2013.

- Nocedal, J. and Wright, S. J. *Numerical optimization*. Springer, 1999.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.
- Petersen, K. B., Pedersen, M. S., et al. The matrix cookbook. *Technical University of Denmark*, 7(15):510, 2008.
- Qi, X., Wang, J., Chen, Y., Shi, Y., and Zhang, L. Lipsformer: Introducing lipschitz continuity to vision transformers. In *The Eleventh International Conference on Learning Representations*, 2023a.
- Qi, X., Wang, J., and Zhang, L. Understanding optimization of deep learning via jacobian matrix and lipschitz constant. *arXiv preprint arXiv:2306.09338*, 2023b.
- Qi, X., Chen, M., Xiao, W., Ye, J., He, Y., Li, C.-G., and Lin, Z. Dnt: a deeply normalized transformer that can be trained by momentum sgd. *arXiv preprint arXiv:2507.17501*, 2025a.
- Qi, X., He, Y., Ye, J., Li, C.-G., Zi, B., Dai, X., Zou, Q., and Xiao, R. Taming transformer without using learning rate warmup. *The Thirteenth International Conference on Learning Representations*, 2025b.
- Qi, X., Ye, J., He, Y., Li, C.-G., Zi, B., Dai, X., Zou, Q., and Xiao, R. Stable-transformer: Towards a stable transformer training, 2025c. URL <https://openreview.net/forum?id=1kRjnNW0gb>.
- Radford, A., Narasimhan, K., Salimans, T., Sutskever, I., et al. Improving language understanding by generative pre-training. 2018.
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I., et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.
- Shazeer, N. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.
- Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., and Roformer, Y. L. Enhanced transformer with rotary position embedding., 2021. DOI: <https://doi.org/10.1016/j.neucom.2023>.
- Team, Q. Qwen technical report. *arXiv preprint arXiv:2309.16609*, 2023.
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023a.
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023b.
- Ulyanov, D., Vedaldi, A., and Lempitsky, V. Instance normalization: The missing ingredient for fast stylization. *arXiv preprint arXiv:1607.08022*, 2016.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- Wang, H., Ma, S., Dong, L., Huang, S., Zhang, D., and Wei, F. Deepnet: Scaling transformers to 1,000 layers. *arXiv preprint arXiv:2203.00555*, 2022.
- Wang, Q., Li, B., Xiao, T., Zhu, J., Li, C., Wong, D. F., and Chao, L. S. Learning deep transformer models for machine translation. *arXiv preprint arXiv:1906.01787*, 2019.
- Wu, Y. and He, K. Group normalization. In *Proceedings of the European conference on computer vision (ECCV)*, pp. 3–19, 2018.
- Xie, Z., Wei, Y., Cao, H., Zhao, C., Deng, C., Li, J., Dai, D., Gao, H., Chang, J., Zhao, L., et al. mhc: Manifold-constrained hyper-connections. *arXiv preprint arXiv:2512.24880*, 2025.
- Zhang, B. and Sennrich, R. Root mean square layer normalization. *Advances in Neural Information Processing Systems*, 32, 2019.
- Zhang, Y., Chen, C., Li, Z., Ding, T., Wu, C., Kingma, D. P., Ye, Y., Luo, Z.-Q., and Sun, R. Adam-mini: Use fewer learning rates to gain more. *arXiv preprint arXiv:2406.16793*, 2024.
- Zhu, J., Chen, X., He, K., LeCun, Y., and Liu, Z. Transformers without normalization. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 14901–14911, 2025.

The derivations of the equations appearing in both the main text and the appendix are available in the following material (Golub & Van Loan, 2013; Horn & Johnson, 2012; Petersen et al., 2008; Magnus & Neudecker, 2019; Nesterov, 1983; 1998; 2013; Boyd & Vandenberghe, 2004; Qi et al., 2025b; 2023b; 2025a).

A. Derivatives of $\nabla_x \ell$ and $\nabla_x^2 \ell$ for SimpleNorm

We consider the mapping

$$\mathbf{y} = \gamma \odot \sqrt{d} \frac{\mathbf{W} \mathbf{x}}{\|\mathbf{W} \mathbf{x}\|_2},$$

where $\gamma \in \mathbb{R}^d$, and $\odot$ denotes elementwise multiplication and $\ell = \ell(\mathbf{y})$ is a scalar loss.

As before, let us define some intermediate variables,

$$\mathbf{z} = \mathbf{W} \mathbf{x}, \quad s = \|\mathbf{z}\|_2, \quad \mathbf{u} = \frac{\mathbf{z}}{s}, \quad \mathbf{P} = \mathbf{I} - \mathbf{u} \mathbf{u}^\top, \quad \mathbf{D} = \text{Diag}(\gamma),$$

where $\mathbf{P}, \mathbf{D}$ are symmetric.

Let us define the normalized and scaled output as

$$\mathbf{y} = \sqrt{d} \mathbf{D} \mathbf{u},$$

and let

$$\mathbf{g}_y := \nabla_y \ell, \quad \mathbf{H}_{yy} := \nabla_{yy}^2 \ell.$$

Let us define the Jacobian $\mathbf{A}$ of the normalization map $\mathbf{u}$ with respect to $\mathbf{z}$ as

$$\mathbf{A} := \frac{\partial \mathbf{u}}{\partial \mathbf{z}} = \frac{1}{s} \mathbf{P},$$

and consequently

$$\frac{\partial \mathbf{y}}{\partial \mathbf{z}} = \sqrt{d} \mathbf{D} \mathbf{A} = \frac{\sqrt{d}}{s} \mathbf{D} \mathbf{P}.$$

We will repeatedly use the chain rule through the path

$$\mathbf{W} \rightarrow \mathbf{z} \rightarrow \mathbf{u} \rightarrow \mathbf{y} \rightarrow \ell, \quad \text{and similarly} \quad \gamma \rightarrow \mathbf{D} \rightarrow \mathbf{y} \rightarrow \ell.$$

(1) First-order derivative. Since $\mathbf{z} = \mathbf{W} \mathbf{x}$, we have

$$d\mathbf{z} = \mathbf{W} d\mathbf{x}.$$

From $\mathbf{u} = \mathbf{z}/s$ and $\mathbf{A} = \partial \mathbf{u} / \partial \mathbf{z}$, it follows that

$$d\mathbf{u} = \mathbf{A} d\mathbf{z} = \frac{1}{s} \mathbf{P} d\mathbf{z}.$$

Therefore,

$$d\mathbf{y} = \sqrt{d} \mathbf{D} d\mathbf{u} = \sqrt{d} \mathbf{D} \mathbf{A} \mathbf{W} d\mathbf{x},$$

so the Jacobian is

$$\mathbf{J}_x^y := \frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \sqrt{d} \mathbf{D} \mathbf{A} \mathbf{W} = \frac{\sqrt{d}}{s} \mathbf{D} \mathbf{P} \mathbf{W}.$$

Applying the chain rule gives

$$\nabla_x \ell = \mathbf{J}_x^y{}^\top \mathbf{g}_y = \sqrt{d} \mathbf{W}^\top \mathbf{A}^\top \mathbf{D}^\top \mathbf{g}_y.$$

Using $\mathbf{A}^\top = \mathbf{A}$ and $\mathbf{D}^\top = \mathbf{D}$, we obtain

$$\nabla_x \ell = \sqrt{d} \mathbf{W}^\top \mathbf{A} \mathbf{D} \mathbf{g}_y = \frac{\sqrt{d}}{s} \mathbf{W}^\top \mathbf{P} \mathbf{D} \mathbf{g}_y. \quad (9)$$

(2) **Second-order derivative.** Differentiating $\nabla_x \ell = \sqrt{d} \mathbf{W}^\top \mathbf{A} \mathbf{D} \mathbf{g}_y$ yields

$$d(\nabla_x \ell) = \sqrt{d} \mathbf{W}^\top (\mathbf{A} \mathbf{D} d\mathbf{g}_y + d\mathbf{A} \mathbf{D} \mathbf{g}_y).$$

This induces the standard decomposition

$$\nabla_x^2 \ell = \mathbf{J}_x^y{}^\top \mathbf{H}_{yy} \mathbf{J}_x^y + \mathbf{C},$$

where the two terms are computed below.

(A) Linear (Gauss–Newton) term.

Since $d\mathbf{g}_y = \mathbf{H}_{yy} d\mathbf{y}$, and $d\mathbf{y} = \mathbf{J}_x^y dx$, we have

$$d\mathbf{g}_y = \mathbf{H}_{yy} \mathbf{J}_x^y dx.$$

Substituting into $\sqrt{d} \mathbf{W}^\top \mathbf{A} \mathbf{D} d\mathbf{g}_y$ gives

$$\sqrt{d} \mathbf{W}^\top \mathbf{A} \mathbf{D} d\mathbf{g}_y = \sqrt{d} \mathbf{W}^\top \mathbf{A} \mathbf{D} \mathbf{H}_{yy} \mathbf{J}_x^y dx = \mathbf{J}_x^y{}^\top \mathbf{H}_{yy} \mathbf{J}_x^y dx.$$

Using $\mathbf{A} = \mathbf{P}/s$ and $\mathbf{J}_x^y = (\sqrt{d}/s) \mathbf{D} \mathbf{P} \mathbf{W}$, we obtain the explicit form

$$\mathbf{J}_x^y{}^\top \mathbf{H}_{yy} \mathbf{J}_x^y = \frac{d}{s^2} \mathbf{W}^\top \mathbf{P} \mathbf{D} \mathbf{H}_{yy} \mathbf{D} \mathbf{P} \mathbf{W}.$$

(B) Curvature (normalization) term.

Recall $\mathbf{A} = \frac{1}{s} \mathbf{P}$. Using $d\mathbf{u} = \mathbf{A} d\mathbf{z}$ and $ds = d\|\mathbf{z}\|_2 = \mathbf{u}^\top d\mathbf{z}$, one obtains

$$d\mathbf{A} = -\frac{1}{s^2} \left((\mathbf{u}^\top d\mathbf{z}) \mathbf{P} + \mathbf{P} d\mathbf{z} \mathbf{u}^\top + \mathbf{u} d\mathbf{z}^\top \mathbf{P} \right).$$

Right-multiplying by $\mathbf{D} \mathbf{g}_y$ gives

$$d\mathbf{A} \mathbf{D} \mathbf{g}_y = -\frac{1}{s^2} \left((\mathbf{u}^\top d\mathbf{z}) \mathbf{P} \mathbf{D} \mathbf{g}_y + \mathbf{P} d\mathbf{z} \mathbf{u}^\top \mathbf{D} \mathbf{g}_y + \mathbf{u} d\mathbf{z}^\top \mathbf{P} \mathbf{D} \mathbf{g}_y \right).$$

Equivalently, pulling the scalar contractions to the left yields the linear map in $d\mathbf{z}$:

$$d\mathbf{A} \mathbf{D} \mathbf{g}_y = -\frac{1}{s^2} \left(\mathbf{P} \mathbf{D} \mathbf{g}_y \mathbf{u}^\top + \mathbf{u}^\top \mathbf{D} \mathbf{g}_y \mathbf{P} + \mathbf{u} \mathbf{g}_y^\top \mathbf{D} \mathbf{P} \right) d\mathbf{z}.$$

Substituting $d\mathbf{z} = \mathbf{W} dx$ and left-multiplying by $\sqrt{d} \mathbf{W}^\top$ gives the curvature term

$$\mathbf{C} = -\frac{\sqrt{d}}{s^2} \mathbf{W}^\top \left(\mathbf{P} \mathbf{D} \mathbf{g}_y \mathbf{u}^\top + \mathbf{u}^\top \mathbf{D} \mathbf{g}_y \mathbf{P} + \mathbf{u} \mathbf{g}_y^\top \mathbf{D} \mathbf{P} \right) \mathbf{W}.$$

(If desired, define $\mathbf{g}_1 := \mathbf{D} \mathbf{g}_y$ to simplify the expression, but we keep the figure’s symbols explicit.)

Combining the linear and curvature terms, the Hessian with respect to $\mathbf{x}$ is

$$\mathbf{H}_{xx} = \nabla_x^2 \ell = \frac{d}{s^2} \mathbf{W}^\top \mathbf{P} \mathbf{D} \mathbf{H}_{yy} \mathbf{D} \mathbf{P} \mathbf{W} - \frac{\sqrt{d}}{s^2} \mathbf{W}^\top \left(\mathbf{P} \mathbf{D} \mathbf{g}_y \mathbf{u}^\top + \mathbf{u}^\top \mathbf{D} \mathbf{g}_y \mathbf{P} + \mathbf{u} \mathbf{g}_y^\top \mathbf{D} \mathbf{P} \right) \mathbf{W}, \quad (10)$$

where $\mathbf{z} = \mathbf{W} \mathbf{x}$, $s = \|\mathbf{z}\|_2$, $\mathbf{u} = \frac{\mathbf{z}}{s}$, $\mathbf{P} = \mathbf{I} - \mathbf{u} \mathbf{u}^\top$, $\mathbf{D} = \text{Diag}(\gamma)$.

B. Derivatives of $\nabla_{\mathbf{W}} \ell$ and $\nabla_{\text{vec}(\mathbf{W})}^2 \ell$ for $\mathbf{y} = \mathbf{W} \mathbf{x}$

We consider the linear mapping

$$\mathbf{y} = \mathbf{W} \mathbf{x},$$

where $\mathbf{W} \in \mathbb{R}^{d \times m}$, $\mathbf{x} \in \mathbb{R}^m$, and $\mathbf{y} \in \mathbb{R}^d$. Let $\ell = \ell(\mathbf{y})$ be a scalar-valued loss function.

We define the first- and second-order derivatives of the loss ℓ with respect to $\mathbf{y}$ as

$$\mathbf{g}_{\mathbf{y}} := \nabla_{\mathbf{y}} \ell(\mathbf{y}) \in \mathbb{R}^d, \quad \mathbf{H}_{\mathbf{y}\mathbf{y}} := \nabla_{\mathbf{y}}^2 \ell(\mathbf{y}) \in \mathbb{R}^{d \times d}.$$

(1) First-order derivative with respect to $\mathbf{W}$. Since $\mathbf{y}$ depends linearly on $\mathbf{W}$, a first-order variation satisfies

$$d\mathbf{y} = d\mathbf{W} \mathbf{x}.$$

Applying the chain rule,

$$d\ell = \mathbf{g}_{\mathbf{y}}^\top d\mathbf{y} = \mathbf{g}_{\mathbf{y}}^\top (d\mathbf{W} \mathbf{x}) = \langle \mathbf{g}_{\mathbf{y}} \mathbf{x}^\top, d\mathbf{W} \rangle_F.$$

Matching coefficients under the Frobenius inner product yields the gradient

$$\nabla_{\mathbf{W}} \ell = \mathbf{g}_{\mathbf{y}} \mathbf{x}^\top. \quad (11)$$

(2) Second-order derivative with respect to $\mathbf{W}$. To express second-order derivatives, it is convenient to vectorize $\mathbf{W}$. Using the standard identity for matrix–vector products,

$$\mathbf{y} = (\mathbf{x}^\top \otimes \mathbf{I}_d) \text{vec}(\mathbf{W}),$$

which makes the dependence of $\mathbf{y}$ on $\text{vec}(\mathbf{W})$ explicit.

Jacobian with respect to $\text{vec}(\mathbf{W})$. From the above expression, the Jacobian of $\mathbf{y}$ with respect to $\text{vec}(\mathbf{W})$ is

$$\mathbf{J}_{\text{vec}(\mathbf{W})}^{\mathbf{y}} \triangleq \frac{\partial \mathbf{y}}{\partial \text{vec}(\mathbf{W})} = \mathbf{x}^\top \otimes \mathbf{I}_d \in \mathbb{R}^{d \times (md)}.$$

Hessian with respect to $\text{vec}(\mathbf{W})$. Because the mapping $\mathbf{W} \mapsto \mathbf{y}$ is linear, it contributes no second-order term. Hence, by the second-order chain rule, the Hessian of the loss with respect to $\text{vec}(\mathbf{W})$ is

$$\nabla_{\text{vec}(\mathbf{W})}^2 \ell = \mathbf{J}_{\text{vec}(\mathbf{W})}^{\mathbf{y}^\top} \mathbf{H}_{\mathbf{y}\mathbf{y}} \mathbf{J}_{\text{vec}(\mathbf{W})}^{\mathbf{y}}.$$

Substituting the explicit Jacobian gives us

$$\nabla_{\text{vec}(\mathbf{W})}^2 \ell = (\mathbf{x} \otimes \mathbf{I}_d) \mathbf{H}_{\mathbf{y}\mathbf{y}} (\mathbf{x}^\top \otimes \mathbf{I}_d).$$

Kronecker-product simplification. Using standard identities of the Kronecker product, the Hessian simplifies to

$$\nabla_{\text{vec}(\mathbf{W})}^2 \ell = (\mathbf{x} \mathbf{x}^\top) \otimes \mathbf{H}_{\mathbf{y}\mathbf{y}}. \quad (12)$$

This result shows that the Hessian with respect to the weight matrix factorizes into a Kronecker product of an input-dependent term $\mathbf{x} \mathbf{x}^\top$ and an output-space curvature term $\mathbf{H}_{\mathbf{y}\mathbf{y}}$. Equivalently, when viewed as a block matrix with $m \times m$ blocks of size $d \times d$, the (i, j) -th block is

$$[\nabla_{\text{vec}(\mathbf{W})}^2 \ell]_{(i,j) \text{ block}} = x_i x_j \mathbf{H}_{\mathbf{y}\mathbf{y}},$$

where x_i denotes the i -th entry of $\mathbf{x}$.

C. Derivatives of $\nabla_\gamma \ell$, $\nabla_\gamma^2 \ell$, $\nabla_W \ell$ and $\nabla_{\text{vec}(W)}^2 \ell$ where $\mathbf{y} = \gamma \odot \sqrt{d} \frac{W\mathbf{x}}{\|W\mathbf{x}\|_2}$.

We consider the mapping

$$\mathbf{y} = \gamma \odot \sqrt{d} \frac{W\mathbf{x}}{\|W\mathbf{x}\|_2},$$

where $\odot$ denotes elementwise multiplication and $\ell = \ell(\mathbf{y})$ is a scalar loss. Define the intermediate variables

$$\mathbf{z} = W\mathbf{x}, \quad s = \|\mathbf{z}\|_2, \quad \mathbf{u} = \frac{\mathbf{z}}{s}, \quad \mathbf{P} = \mathbf{I} - \mathbf{u}\mathbf{u}^\top, \quad \mathbf{D} = \text{Diag}(\gamma).$$

We have

$$\mathbf{y} = \sqrt{d} \mathbf{D} \mathbf{u}, \quad \ell = \ell(\mathbf{y}), \quad \mathbf{g}_\mathbf{y} := \nabla_\mathbf{y} \ell, \quad \mathbf{H}_{\mathbf{y}\mathbf{y}} := \nabla_{\mathbf{y}\mathbf{y}}^2 \ell.$$

Remind two useful Jacobian matrices,

$$\frac{\partial \mathbf{u}}{\partial \mathbf{z}} = \mathbf{A} = \frac{1}{s} \mathbf{P}, \quad \frac{\partial \mathbf{y}}{\partial \mathbf{z}} = \sqrt{d} \mathbf{D} \mathbf{A} = \frac{\sqrt{d}}{s} \mathbf{D} \mathbf{P}.$$

We will repeatedly use the chain rule through the path $W \rightarrow \mathbf{z} \rightarrow \mathbf{u} \rightarrow \mathbf{y} \rightarrow \ell$, and similarly $\gamma \rightarrow \mathbf{D} \rightarrow \mathbf{y} \rightarrow \ell$.

C.1. Part I: derivatives w.r.t. γ

(1) First-order derivative. Since $\mathbf{y}_i = \sqrt{d} \gamma_i u_i$, the Jacobian of $\mathbf{y}$ w.r.t. γ is diagonal:

$$\frac{\partial \mathbf{y}}{\partial \gamma} = \sqrt{d} \text{Diag}(\mathbf{u}).$$

Applying the chain rule $\mathbf{g}_\gamma = \left(\frac{\partial \mathbf{y}}{\partial \gamma}\right)^\top \mathbf{g}_\mathbf{y}$, we obtain

$$\mathbf{g}_\gamma = \nabla_\gamma \ell = \sqrt{d} \text{Diag}(\mathbf{u}) \mathbf{g}_\mathbf{y} = \sqrt{d} (\mathbf{u} \odot \mathbf{g}_\mathbf{y}). \quad (13)$$

(2) Second-order derivative. Because $\mathbf{y}$ is linear in γ , $\frac{\partial^2 \mathbf{y}}{\partial \gamma^2} = 0$, hence the Hessian comes purely from $\mathbf{H}_{\mathbf{y}\mathbf{y}}$:

$$\mathbf{H}_{\gamma\gamma} = \nabla_{\gamma\gamma}^2 \ell = \left(\frac{\partial \mathbf{y}}{\partial \gamma}\right)^\top \mathbf{H}_{\mathbf{y}\mathbf{y}} \left(\frac{\partial \mathbf{y}}{\partial \gamma}\right) = d \text{Diag}(\mathbf{u}) \mathbf{H}_{\mathbf{y}\mathbf{y}} \text{Diag}(\mathbf{u}). \quad (14)$$

C.2. Part II: derivatives w.r.t. W

(1) First-order derivative. Step 1 ($\mathbf{y} \rightarrow \mathbf{z}$): gradient w.r.t. $\mathbf{z}$

By the chain rule,

$$\mathbf{g}_\mathbf{z} = \left(\frac{\partial \mathbf{y}}{\partial \mathbf{z}}\right)^\top \mathbf{g}_\mathbf{y} = \left(\frac{\sqrt{d}}{s} \mathbf{D} \mathbf{P}\right)^\top \mathbf{g}_\mathbf{y}.$$

Using $\mathbf{P}^\top = \mathbf{P}$ and $\mathbf{D}^\top = \mathbf{D}$, this simplifies to

$$\mathbf{g}_\mathbf{z} = \frac{\sqrt{d}}{s} \mathbf{P} \mathbf{D} \mathbf{g}_\mathbf{y}.$$

Step 2 ($\mathbf{z} = W\mathbf{x} \rightarrow W$): gradient w.r.t. W

We write the differential $d\mathbf{z} = dW \mathbf{x}$, so

$$d\ell = \mathbf{g}_\mathbf{z}^\top d\mathbf{z} = \mathbf{g}_\mathbf{z}^\top (dW \mathbf{x}) = \langle \mathbf{g}_\mathbf{z} \mathbf{x}^\top, dW \rangle_F.$$

Matching coefficients under the Frobenius inner product yields

$$\mathbf{g}_W = \nabla_W \ell = \mathbf{g}_\mathbf{z} \mathbf{x}^\top = \frac{\sqrt{d}}{s} (\mathbf{P} \mathbf{D} \mathbf{g}_\mathbf{y}) \mathbf{x}^\top. \quad (15)$$

If one needs the vectorized mapping,

$$\text{vec}(\mathbf{z}) = \text{vec}(W\mathbf{x}) = (\mathbf{x}^\top \otimes \mathbf{I}) \text{vec}(W).$$

(2) Second-order derivative. We first derive $H_{zz} := \nabla_{zz}^2 \ell$, then lift it to $\text{vec}(\mathbf{W})$.

Step 1: Hessian w.r.t. $\mathbf{z}$

The second-order chain rule gives

$$H_{zz} = \left(\frac{\partial \mathbf{y}}{\partial \mathbf{z}} \right)^\top H_{yy} \left(\frac{\partial \mathbf{y}}{\partial \mathbf{z}} \right) + \sum_{k=1}^d (g_y)_k \nabla_{zz}^2 y_k.$$

The first term is the Gauss–Newton part:

$$\left(\frac{\partial \mathbf{y}}{\partial \mathbf{z}} \right)^\top H_{yy} \left(\frac{\partial \mathbf{y}}{\partial \mathbf{z}} \right) = \frac{d}{s^2} \mathbf{P} \mathbf{D} H_{yy} \mathbf{D} \mathbf{P}.$$

For the second term, we use the bilinear form of the normalization Hessian:

$$\nabla^2 \mathbf{u}(\mathbf{z})[\mathbf{a}, \mathbf{b}] = -\frac{1}{s^2} \left((\mathbf{u}^\top \mathbf{a}) \mathbf{P} \mathbf{b} + (\mathbf{u}^\top \mathbf{b}) \mathbf{P} \mathbf{a} + (\mathbf{a}^\top \mathbf{P} \mathbf{b}) \mathbf{u} \right).$$

Since $\mathbf{y} = \sqrt{d} \mathbf{D} \mathbf{u}$,

$$\sum_{k=1}^d (g_y)_k \nabla^2 y_k[\mathbf{a}, \mathbf{b}] = \mathbf{g}_y^\top \nabla^2 \mathbf{y}[\mathbf{a}, \mathbf{b}] = \sqrt{d} (\mathbf{D} \mathbf{g}_y)^\top \nabla^2 \mathbf{u}[\mathbf{a}, \mathbf{b}].$$

Substituting gives

$$\sum_{k=1}^d (g_y)_k \nabla^2 y_k[\mathbf{a}, \mathbf{b}] = -\frac{\sqrt{d}}{s^2} \left((\mathbf{u}^\top \mathbf{a}) (\mathbf{D} \mathbf{g}_y)^\top \mathbf{P} \mathbf{b} + (\mathbf{u}^\top \mathbf{b}) (\mathbf{D} \mathbf{g}_y)^\top \mathbf{P} \mathbf{a} + (\mathbf{u}^\top \mathbf{D} \mathbf{g}_y) \mathbf{a}^\top \mathbf{P} \mathbf{b} \right).$$

Equivalently, the associated matrix form is

$$\sum_{k=1}^d (g_y)_k \nabla_{zz}^2 y_k = -\frac{\sqrt{d}}{s^2} \left(\mathbf{P} \mathbf{D} \mathbf{g}_y \mathbf{u}^\top + \mathbf{u}^\top \mathbf{D} \mathbf{g}_y \mathbf{P} + \mathbf{u} \mathbf{g}_y^\top \mathbf{D} \mathbf{P} \right).$$

Combining both terms,

$$H_{zz} = \frac{d}{s^2} \mathbf{P} \mathbf{D} H_{yy} \mathbf{D} \mathbf{P} - \frac{\sqrt{d}}{s^2} \left(\mathbf{P} \mathbf{D} \mathbf{g}_y \mathbf{u}^\top + \mathbf{u}^\top \mathbf{D} \mathbf{g}_y \mathbf{P} + \mathbf{u} \mathbf{g}_y^\top \mathbf{D} \mathbf{P} \right), \quad s = \|\mathbf{W} \mathbf{x}\|_2.$$

Using $\text{vec}(\mathbf{z}) = (\mathbf{x}^\top \otimes \mathbf{I}) \text{vec}(\mathbf{W})$,

$$\begin{aligned} H_{\text{vec}(\mathbf{W}) \text{vec}(\mathbf{W})} &= (\mathbf{x} \mathbf{x}^\top) \otimes H_{zz} \\ &= (\mathbf{x} \mathbf{x}^\top) \otimes \left[\frac{d}{s^2} \mathbf{P} \mathbf{D} H_{yy} \mathbf{D} \mathbf{P} - \frac{\sqrt{d}}{s^2} \left(\mathbf{P} \mathbf{D} \mathbf{g}_y \mathbf{u}^\top + \mathbf{u}^\top \mathbf{D} \mathbf{g}_y \mathbf{P} + \mathbf{u} \mathbf{g}_y^\top \mathbf{D} \mathbf{P} \right) \right]. \end{aligned} \quad (16)$$

D. Proof of Theorem 4.1

Proof. Recall SimpleNorm:

$$\mathbf{y} = \gamma \odot \sqrt{d} \frac{\mathbf{W}\mathbf{x}}{\|\mathbf{W}\mathbf{x}\|_2}, \quad \mathbf{z} = \mathbf{W}\mathbf{x}, \quad s := \|\mathbf{z}\|_2, \quad \mathbf{u} := \frac{\mathbf{z}}{s}, \quad \mathbf{P} := \mathbf{I} - \mathbf{u}\mathbf{u}^\top.$$

Assume $\mathbf{D} = \text{Diag}(\gamma) = \mathbf{I}$. Let $\ell = \ell(\mathbf{y})$ and define

$$\mathbf{g}_\mathbf{y} := \nabla_\mathbf{y} \ell(\mathbf{y}) \in \mathbb{R}^d, \quad \mathbf{H}_{\mathbf{y}\mathbf{y}} := \nabla_{\mathbf{y}\mathbf{y}}^2 \ell(\mathbf{y}) \in \mathbb{R}^{d \times d}.$$

By the chain rule, we have the decomposition:

$$\mathbf{H}_{\mathbf{x}\mathbf{x}} = \nabla_\mathbf{x}^2 \ell = \underbrace{\mathbf{J}_\mathbf{x}^\top \mathbf{H}_{\mathbf{y}\mathbf{y}} \mathbf{J}_\mathbf{x}}_{\text{Gauss-Newton term}} + \underbrace{\mathbf{C}}_{\text{curvature term}} \quad (17)$$

Here the Jacobian is

$$\mathbf{J}_\mathbf{x}^\top = \frac{\sqrt{d}}{s} \mathbf{D} \mathbf{P} \mathbf{W} = \frac{\sqrt{d}}{s} \mathbf{P} \mathbf{W},$$

and therefore the Gauss-Newton term is

$$\mathbf{L} = (\mathbf{J}_\mathbf{x}^\top)^\top \mathbf{H}_{\mathbf{y}\mathbf{y}} \mathbf{J}_\mathbf{x}^\top = \frac{d}{s^2} \mathbf{W}^\top \mathbf{P} \mathbf{H}_{\mathbf{y}\mathbf{y}} \mathbf{P} \mathbf{W}. \quad (18)$$

The curvature term (with the condition $\mathbf{D} = \mathbf{I}$) can be written as

$$\mathbf{C} = -\frac{\sqrt{d}}{s^2} \mathbf{W}^\top \left((\mathbf{P} \mathbf{g}_\mathbf{y}) \mathbf{u}^\top + (\mathbf{u}^\top \mathbf{g}_\mathbf{y}) \mathbf{P} + \mathbf{u} (\mathbf{g}_\mathbf{y}^\top \mathbf{P}) \right) \mathbf{W}, \quad (19)$$

where $\mathbf{u}^\top \mathbf{g}_\mathbf{y}$ is a scalar.

Bounding the Gauss-Newton term. Define

$$\kappa := \frac{\sqrt{d}}{\|\mathbf{W}\mathbf{x}\|_2} \|\mathbf{W}\|_2$$

We show that $\kappa = \Theta(1)$ under high effective rank and typical input. Let $\mathbf{x} = \sqrt{d} \xi$ where ξ is isotropic (e.g., uniform on the sphere or subgaussian). Then

$$\mathbb{E} \|\mathbf{W}\mathbf{x}\|_2^2 = d \mathbb{E} \xi^\top \mathbf{W}^\top \mathbf{W} \xi = \|\mathbf{W}\|_F^2,$$

and by standard concentration for quadratic forms,

$$\|\mathbf{W}\mathbf{x}\|_2^2 \asymp \|\mathbf{W}\|_F^2 \implies \|\mathbf{W}\mathbf{x}\|_2 \asymp \|\mathbf{W}\|_F \quad \text{with high probability.}$$

Thus

$$\kappa = \frac{\sqrt{d}}{\|\mathbf{W}\mathbf{x}\|_2} \|\mathbf{W}\|_2 \asymp \sqrt{d} \frac{\|\mathbf{W}\|_2}{\|\mathbf{W}\|_F} = \sqrt{\frac{d}{r_{\text{eff}}(\mathbf{W})}}, \quad r_{\text{eff}}(\mathbf{W}) := \frac{\|\mathbf{W}\|_F^2}{\|\mathbf{W}\|_2^2}.$$

If $r_{\text{eff}}(\mathbf{W}) \asymp cd$, then $\kappa \asymp 1/\sqrt{c} = \Theta(1)$ with high probability.

Additionally, define

$$\tau := \frac{\|\widetilde{\mathbf{W}}^\top \mathbf{P} \mathbf{H}_{\mathbf{y}\mathbf{y}} \mathbf{P} \widetilde{\mathbf{W}}\|_2}{\|\mathbf{H}_{\mathbf{y}\mathbf{y}}\|_2} \in [0, 1], \quad \widetilde{\mathbf{W}} := \frac{\mathbf{W}}{\|\mathbf{W}\|_2}.$$

Under the theorem's "non-pathological alignment" assumption (i.e., the dominant spectral modes of $\mathbf{H}_{\mathbf{y}\mathbf{y}}$ are not eliminated by projecting out $\text{span}(\mathbf{u})$ and are represented in the range of $\mathbf{W}$, which has high effective rank), we have $\tau = \Theta(1)$.

Now, it follows that

$$\mathbf{L} = \frac{d}{\|\mathbf{W}\mathbf{x}\|_2^2} \mathbf{W}^\top \mathbf{P} \mathbf{H}_{\mathbf{y}\mathbf{y}} \mathbf{P} \mathbf{W} = \frac{\kappa^2}{\|\mathbf{W}\|_2^2} \mathbf{W}^\top \mathbf{P} \mathbf{H}_{\mathbf{y}\mathbf{y}} \mathbf{P} \mathbf{W} = \kappa^2 \widetilde{\mathbf{W}}^\top \mathbf{P} \mathbf{H}_{\mathbf{y}\mathbf{y}} \mathbf{P} \widetilde{\mathbf{W}}$$

and

$$\|\mathbf{L}\|_2 = \tau \kappa^2 \|\mathbf{H}_{\mathbf{y}\mathbf{y}}\|_2 \quad \text{where } \tau = \Theta(1) \text{ and } \kappa = \Theta(1) \quad [\text{w.h.p}] \quad (20)$$

Bounding the curvature term. From Equation 19 and submultiplicativity,

$$\|C\|_2 \leq \frac{\sqrt{d}}{s^2} \|W\|_2^2 \left\| (Pg_y)u^\top + (u^\top g_y)P + u(g_y^\top P) \right\|_2.$$

Now use $\|u\|_2 = 1$, $\|P\|_2 = 1$, and $\|Pg_y\|_2 \leq \|g_y\|_2$:

$$\|(Pg_y)u^\top\|_2 = \|Pg_y\|_2 \|u\|_2 \leq \|g_y\|_2,$$

$$\|(u^\top g_y)P\|_2 = |u^\top g_y| \|P\|_2 \leq \|g_y\|_2,$$

$$\|u(g_y^\top P)\|_2 = \|u\|_2 \|Pg_y\|_2 \leq \|g_y\|_2.$$

By triangle inequality, the middle norm is at most $3\|g_y\|_2$, hence

$$\|C\|_2 \leq \frac{3\sqrt{d}}{\|Wx\|_2^2} \|W\|_2^2 \|g_y\|_2 = \frac{3\kappa^2}{\sqrt{d}} \|g_y\|_2. \quad (21)$$

Dominance of L over C . Combining (20) and (21),

$$\frac{\|C\|_2}{\|L\|_2} \leq \frac{3}{\tau\sqrt{d}} \frac{\|g_y\|_2}{\|H_{yy}\|_2}.$$

Assuming $\tau = \Theta(1)$ (non-pathological alignment) and $\|g_y\|_2/\|H_{yy}\|_2 = O(1)$ (bounded gradient-to-curvature ratio, as stated in the theorem assumptions), we obtain

$$\frac{\|C\|_2}{\|L\|_2} = O(d^{-1/2}),$$

so in high dimension $\|L\|_2 \gg \|C\|_2$ with high probability. Therefore the Gauss–Newton term dominates H_{xx} in typical non-pathological regimes.

This completes the proof of Theorem 4.1. □

E. Proof of Theorem 4.2

Proof. Provided $\ell = \ell(\mathbf{y})$ is twice differentiable. Denote

$$\mathbf{g}_{\mathbf{y}} := \nabla_{\mathbf{y}} \ell, \quad \mathbf{H}_{\mathbf{y}\mathbf{y}} := \nabla_{\mathbf{y}\mathbf{y}}^2 \ell,$$

We have the two mappings:

Linear: $\mathbf{y}_1 = \mathbf{W}_1 \mathbf{x}, \quad \mathbf{H}_{\mathbf{x}\mathbf{x}}^{\text{lin}} = \mathbf{W}_1^\top \mathbf{H}_{\mathbf{y}_1 \mathbf{y}_1} \mathbf{W}_1.$

SimpleNorm: $\mathbf{y}_2 = \mathbf{D} \frac{\sqrt{d} \mathbf{W}_2 \mathbf{x}}{\|\mathbf{W}_2 \mathbf{x}\|_2},$ where $\mathbf{D} = \text{Diag}(\gamma)$ and define

$$\mathbf{z} = \mathbf{W}_2 \mathbf{x}, \quad s = \|\mathbf{z}\|_2, \quad \mathbf{u} = \mathbf{z}/s, \quad \mathbf{P} = \mathbf{I} - \mathbf{u}\mathbf{u}^\top.$$

The Hessian w.r.t. $\mathbf{x}$ admits the standard decomposition

$$\mathbf{H}_{\mathbf{x}\mathbf{x}}^{\text{sn}} = \mathbf{L} + \mathbf{C}, \quad \mathbf{L} = \mathbf{J}_{\mathbf{x}}^{\mathbf{y}_2 \top} \mathbf{H}_{\mathbf{y}_2 \mathbf{y}_2} \mathbf{J}_{\mathbf{x}}^{\mathbf{y}_2},$$

where $\mathbf{J}_{\mathbf{x}}^{\mathbf{y}_2}$ is the Jacobian of $\mathbf{y}_2$ w.r.t. $\mathbf{x}$, and $\mathbf{C}$ is the curvature term induced by the normalization.

Assuming the high-dimensional conditions stated in Theorem 4.1 hold ($\|\mathbf{x}\|_2 = \sqrt{d}$, $\mathbf{D} = \mathbf{I}$, $\|\mathbf{P}\|_2 = 1$, $\mathbf{W}_2$ has high effective rank, no pathological alignment, and $\|\mathbf{g}_{\mathbf{y}}\|_2/\|\mathbf{H}_{\mathbf{y}\mathbf{y}}\|_2 = O(1)$), Theorem 4.1 shows that the Gauss-Newton term dominates the curvature term, i.e., $\|\mathbf{L}\|_2 \gg \|\mathbf{C}\|_2$. Therefore, we obtain the approximation $\|\mathbf{H}_{\mathbf{x}\mathbf{x}}^{\text{sn}}\|_2 \approx \|\mathbf{L}\|_2$.

We now show an integral result: the spectral norm of $\mathbf{H}_{\mathbf{x}\mathbf{x}}^{\text{lin}}$ is directly proportional to the spectral norm of the weight matrix $\mathbf{W}$ whereas the spectral norm of $\mathbf{H}_{\mathbf{x}\mathbf{x}}^{\text{sn}}$ is independent of the spectral norm of the weight matrix. In the following, we assume $\mathbf{W} = \mathbf{W}_1 = \mathbf{W}_2$ and $\mathbf{H}_{\mathbf{y}_1 \mathbf{y}_1} = \mathbf{H}_{\mathbf{y}_2 \mathbf{y}_2} := \mathbf{H}_{\mathbf{y}\mathbf{y}}$.

Define $\widetilde{\mathbf{W}} := \frac{\mathbf{W}}{\alpha}$ where $\alpha := \|\mathbf{W}\|_2$. Consequently, $\|\widetilde{\mathbf{W}}\|_2 = 1$ and $\mathbf{W} = \alpha \widetilde{\mathbf{W}}$. Now, treating $\mathbf{H}_{\mathbf{x}\mathbf{x}}^{\text{lin}}$ and $\mathbf{H}_{\mathbf{x}\mathbf{x}}^{\text{sn}}$ as functions of α , it follows that:

$$\|\mathbf{H}_{\mathbf{x}\mathbf{x}}^{\text{lin}}(\alpha)\|_2 = \|\mathbf{W}_1^\top \mathbf{H}_{\mathbf{y}_1 \mathbf{y}_1} \mathbf{W}_1\|_2 = \alpha^2 \|\widetilde{\mathbf{W}}^\top \mathbf{H}_{\mathbf{y}\mathbf{y}} \widetilde{\mathbf{W}}\|_2 \quad (22)$$

and

$$\|\mathbf{H}_{\mathbf{x}\mathbf{x}}^{\text{sn}}(\alpha)\|_2 \approx \|\mathbf{L}(\alpha)\|_2 = \frac{d}{\|\mathbf{W}\mathbf{x}\|_2^2} \|\mathbf{W}^\top \mathbf{P} \mathbf{D} \mathbf{H}_{\mathbf{y}\mathbf{y}} \mathbf{D} \mathbf{P} \mathbf{W}\|_2 = \frac{d}{\|\widetilde{\mathbf{W}}\mathbf{x}\|_2^2} \|\widetilde{\mathbf{W}}^\top \mathbf{P} \mathbf{H}_{\mathbf{y}\mathbf{y}} \mathbf{P} \widetilde{\mathbf{W}}\|_2 \quad (23)$$

Note that $\mathbf{P} = \mathbf{I} - \mathbf{u}\mathbf{u}^\top$ is independent of α :

$$\mathbf{u}(\alpha) = \frac{\mathbf{W}\mathbf{x}}{\|\mathbf{W}\mathbf{x}\|_2} = \frac{\alpha \widetilde{\mathbf{W}}\mathbf{x}}{\|\alpha \widetilde{\mathbf{W}}\mathbf{x}\|_2} = \frac{\widetilde{\mathbf{W}}\mathbf{x}}{\|\widetilde{\mathbf{W}}\mathbf{x}\|_2}, \quad \implies \quad \mathbf{P}(\alpha) = \mathbf{I} - \mathbf{u}(\alpha)\mathbf{u}(\alpha)^\top = \mathbf{P}(1).$$

Therefore, $\|\mathbf{H}_{\mathbf{x}\mathbf{x}}^{\text{lin}}(\alpha)\|_2$ depends quadratically on α whereas $\|\mathbf{H}_{\mathbf{x}\mathbf{x}}^{\text{sn}}(\alpha)\|_2$ is *scale-invariant* in α .

Next, we compare the magnitudes of the two Hessians. First, recall the definition of κ provided in Theorem 4.1:

$$\kappa := \frac{\sqrt{d}}{\|\mathbf{W}\mathbf{x}\|_2} \|\mathbf{W}\|_2 = \frac{\sqrt{d}}{\alpha \|\widetilde{\mathbf{W}}\mathbf{x}\|_2} \alpha \|\widetilde{\mathbf{W}}\|_2 = \frac{\sqrt{d}}{\|\widetilde{\mathbf{W}}\mathbf{x}\|_2}$$

By definition, $\kappa^2 = \frac{d}{\|\widetilde{\mathbf{W}}\mathbf{x}\|_2^2}$, and Theorem 4.1 implies $\kappa^2 = \Theta(1)$ w.h.p.

Moreover, since $\|\widetilde{\mathbf{W}}\|_2 = \|\mathbf{P}\|_2 = 1$,

$$\|\mathbf{H}_{\mathbf{x}\mathbf{x}}^{\text{sn}}(\alpha)\|_2 \approx \|\mathbf{L}\|_2 = \kappa^2 \|\widetilde{\mathbf{W}}^\top \mathbf{P} \mathbf{H}_{\mathbf{y}\mathbf{y}} \mathbf{P} \widetilde{\mathbf{W}}\|_2 \leq \kappa^2 \|\mathbf{H}_{\mathbf{y}\mathbf{y}}\|_2 \quad (24)$$

For the linear module, there exists a constant $c_{\text{lin}} > 0$ such that

$$\|\mathbf{H}_{xx}^{\text{lin}}(\alpha)\|_2 = \alpha^2 \|\widetilde{\mathbf{W}}^\top \mathbf{H}_{yy} \widetilde{\mathbf{W}}\|_2 \geq \alpha^2 c_{\text{lin}} \|\mathbf{H}_{yy}\|_2.$$

We assume $\text{Range}(\widetilde{\mathbf{W}})$ is not adversarially aligned with the leading eigenspace of $\mathbf{H}_{yy}$; in particular the overlap is constant-order, so the visibility constant satisfies $c_{\text{lin}} = \Theta(1)$ and does not decay with d .

Combining all the aforementioned results, the final ratio between the two Hessians satisfies:

$$\frac{\|\mathbf{H}_{xx}^{\text{lin}}(\alpha)\|_2}{\|\mathbf{H}_{xx}^{\text{sn}}(\alpha)\|_2} \geq \frac{\alpha^2 \|\widetilde{\mathbf{W}}^\top \mathbf{H}_{yy} \widetilde{\mathbf{W}}\|_2}{\kappa^2 \|\mathbf{H}_{yy}\|_2} \geq \alpha^2 \frac{c_{\text{lin}}}{\kappa^2}$$

Empirically, $\alpha = \|\mathbf{W}\|_2$ typically grows to tens or hundreds during training, so $\alpha^2 c_{\text{lin}} \gg \kappa^2$ (recall $\kappa = \Theta(1)$ w.h.p.). Hence:

$$\|\mathbf{H}_{xx}^{\text{lin}}\|_2 \gg \|\mathbf{H}_{xx}^{\text{sn}}\|_2 \quad (\text{with high probability}).$$

In summary, in non-pathological cases, the gradient Lipschitz constant of $\|\mathbf{H}_{xx}^{\text{lin}}\|_2$ is much larger than the gradient Lipschitz constant of $\|\mathbf{H}_{xx}^{\text{sn}}\|_2$.

This completes the proof of [Theorem 4.2](#).

□

F. Empirical Support for Learned RMSNorm Gain Parameter Assumption

[Theorem 4.1](#) is stated under the simplifying assumption $D = \text{Diag}(\gamma) = I$. This assumption is merely used to make the main curvature mechanism transparent, and should not be interpreted as requiring the learned RMSNorm gain parameters to be exactly equal to one in practice. What is needed for the scale-control argument is that the entries of γ remain bounded by a moderate constant, so that the gain parameters do not dominate the normalization effect.

In practice, this is generally upheld, as shown in [Figure 9](#), where we record the learned RMSNorm gain parameters across all layers for Llama2-7B, Llama2-7B with QKNorm, and SimpleGPT-7B.

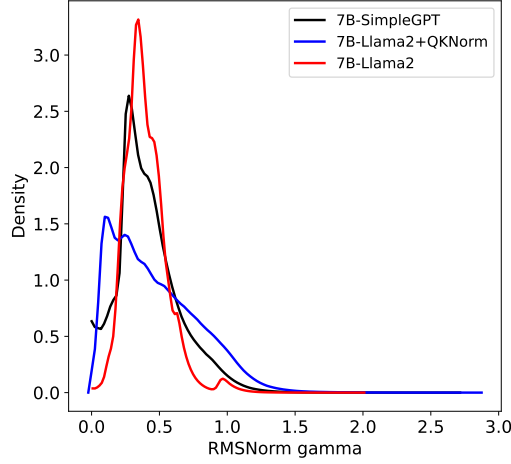


Figure 9. Distribution of learned RMSNorm gain parameters γ across Llama2-7B, Llama2-7B with QKNorm, and SimpleGPT-7B.

G. Detailed Experimental Settings

Our **SimpleGPT** models are built on Llama2, Llama3, and nanoGPT (a GPT-2 implementation). We apply the SimpleNorm operator to all Transformer blocks except the embedding layer and classification layer. Our implementations are based on the Adam-mini ¹ (Zhang et al., 2024) and nanoGPT ² (Karpathy, 2022). Below we briefly describe the main architectural and training settings for each backbone.

nanoGPT is a lightweight and efficient implementation of the GPT-2 architecture. It uses the GELU activation function and a byte-pair encoding (BPE) tokenizer (Gage, 1994) consistent with GPT-2 (Radford et al., 2019), with an expanded vocabulary size of 50,257 tokens. 2,000 steps are used for learning rate warmup. Our training data on nanoGPT models is OpenWebText.

Llama2 adopts the SwiGLU (Shazeer, 2020) activation function in the feed-forward networks, which improves expressivity and parameter efficiency. Positional information is encoded using Rotary Positional Embeddings (RoPE) (Su et al., 2023). Llama2 also introduces Grouped-Query Attention (GQA) to reduce inference-time memory usage and computational cost. The model uses a SentencePiece-based BPE tokenizer with a vocabulary size of 32K tokens. In our experiments, 1% of the total steps are allocated for learning rate warmup. Our training data on Llama2 models is C4.

Llama3 follows the dense Transformer design of Llama2, while introducing several targeted changes. It continues to use GQA with eight key-value heads to improve decoding efficiency and reduce key-value cache size. A major difference lies in the tokenizer: Llama3 adopts a significantly larger vocabulary of 128K tokens, combining tokens from the tiktoken tokenizer with additional multilingual tokens, which improves compression rates and language coverage. To better support long contexts, the RoPE base frequency is increased to 500,000. In our experiments, 1% of the total steps are allocated for learning rate warmup. Our training data on Llama3 models is C4.

Across all experiments, we adopt the AdamW optimizer (Kingma & Ba, 2014; Loshchilov & Hutter, 2019) with $\beta_1 = 0.9$ and $\beta_2 = 0.95$. Since we know that weight decay is associated with the learning rate, and our method permits the use of larger learning rates, we accordingly adjust the weight decay. Unless otherwise stated, a weight decay value of 0.1 is used throughout our experiments. Additional hyperparameter configurations are summarized in Table 2.

All models are trained using PyTorch (Paszke et al., 2019) with bfloat16 precision on A800 GPUs. We employ a cosine learning rate schedule for all training runs.

H. Parameters and configurations of SimpleGPT

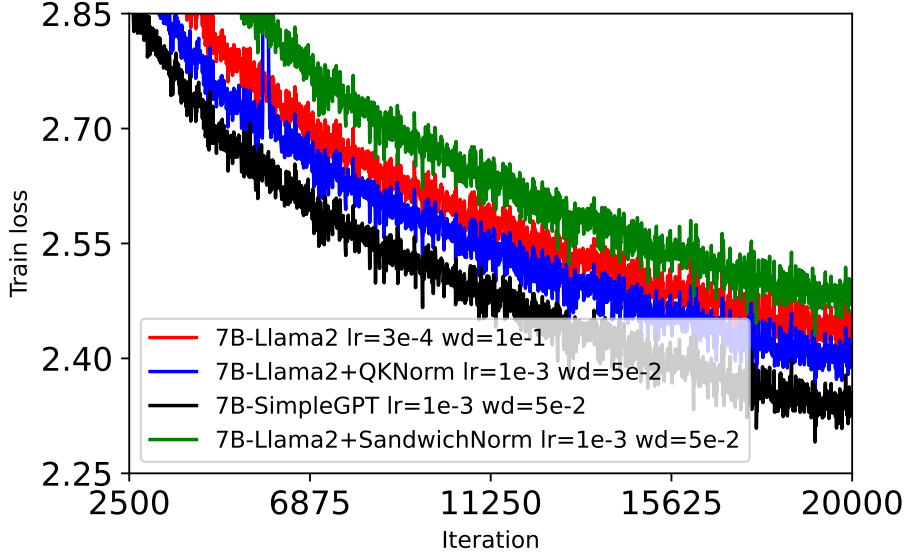
Table 2. Model configurations for different scales of SimpleGPT. The 1B and 7B models are based on Llama2, the 8B model is based on Llama3, and the 1.4B model is based on nanoGPT.

	SimpleGPT-1B	SimpleGPT-7B	SimpleGPT-8B	SimpleGPT-1.4B
Origin from	Llama2	Llama2	Llama3	nanoGPT(GPT2)
Layers	18	32	32	48
Model Dimension	2,048	4,096	4,096	1,536
FFN Dimension	5,632	11,008	14,336	6,144
Attention Heads	16	32	32	24
Key / Value Heads	16	32	8	24
Activation Function	SwiGLU	SwiGLU	SwiGLU	GeLU
Vocabulary Size	32,000	32,000	128,000	50,304
Positional Embeddings (RoPE)	$\theta = 10,000$	$\theta = 10,000$	$\theta = 500,000$	No
Batch Size	512×256	2048×192	2048×192	1024×512
Training Steps	200K	20K/40K/60K	20K	100K
Warmup Steps	1%	1%	1%	2000

¹<https://github.com/zyushun/Adam-mini>

²<https://github.com/karpathy/nanoGPT>

I. Comparison with SandwichNorm



7B-Llama2 lr=3e-4 wd=1e-1: last = 2.431

7B-Llama2+QKNorm lr=1e-3 wd=5e-2: last = 2.393

7B-SimpleGPT lr=1e-3 wd=5e-2: last = 2.331

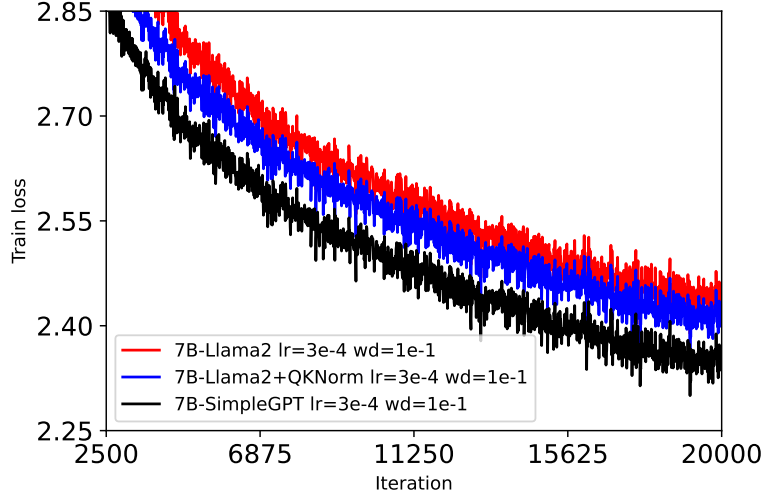
7B-Llama2+SandwichNorm lr=1e-3 wd=5e-2: last = 2.471

Figure 10. Comparison among Llama2-7B, Llama2-7B with QKNorm, Llama2-7B with SandwichNorm, and SimpleGPT-7B.

To examine whether the gains of SimpleGPT come merely from adding more normalization layers, we compare against SandwichNorm (Ding et al., 2021), which inserts normalization around Transformer sublayers. As shown in Figure 10, SimpleGPT-7B achieves lower training loss than Llama2-7B with SandwichNorm. This suggests that the improvement is not simply due to increasing the number of normalization operations, but is more closely related to the SimpleNorm placement principle of normalizing immediately after linear mappings.

J. More experiments on SimpleGPT-7B

Furthermore, we evaluate the SimpleGPT-7B models using different learning rates or weight decay values. Results are shown in Figure 11 and Figure 12.

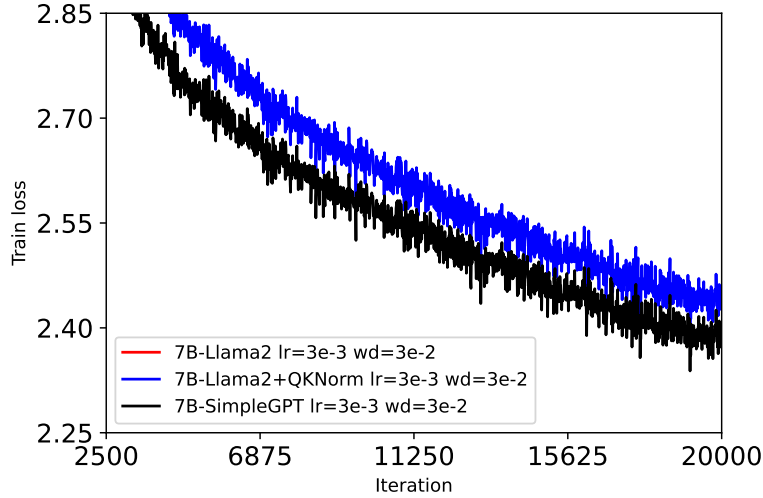


7B-Llama2 lr=3e-4 wd=1e-1: last = 2.431

7B-Llama2+QKNorm lr=3e-4 wd=1e-1: last = 2.404

7B-SimpleGPT lr=3e-4 wd=1e-1: last = 2.340

Figure 11. The training loss curves of Llama2-7B, Llama2-7B with QKNorm and SimpleGPT-7B with learning rate 3e-4 and weight decay 0.1.



7B-Llama2 lr=3e-3 wd=3e-2: last = 4.579

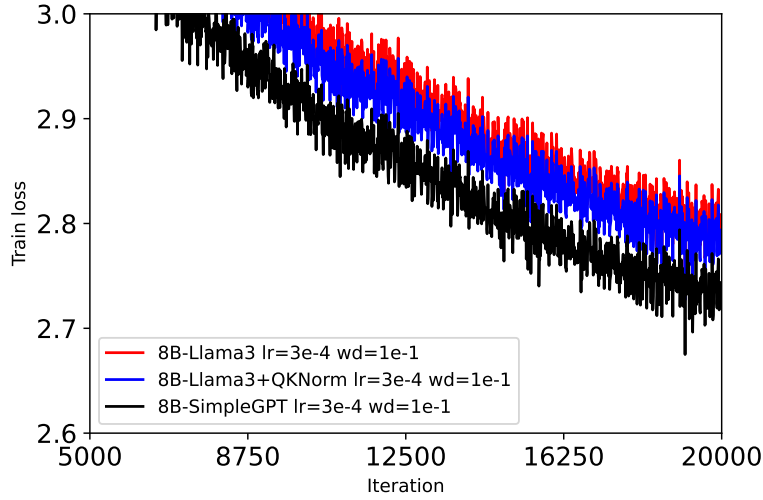
7B-Llama2+QKNorm lr=3e-3 wd=3e-2: last = 2.430

7B-SimpleGPT lr=3e-3 wd=3e-2: last = 2.377

Figure 12. The training loss curves of Llama2-7B, Llama2-7B with QKNorm and SimpleGPT-7B with learning rate 3e-3 and weight decay 0.03.

K. More experiments on SimpleGPT-8B

Furthermore, we evaluate the SimpleGPT-8B models using different learning rates or weight decay values. Results are shown in Figure 13 and Figure 14.

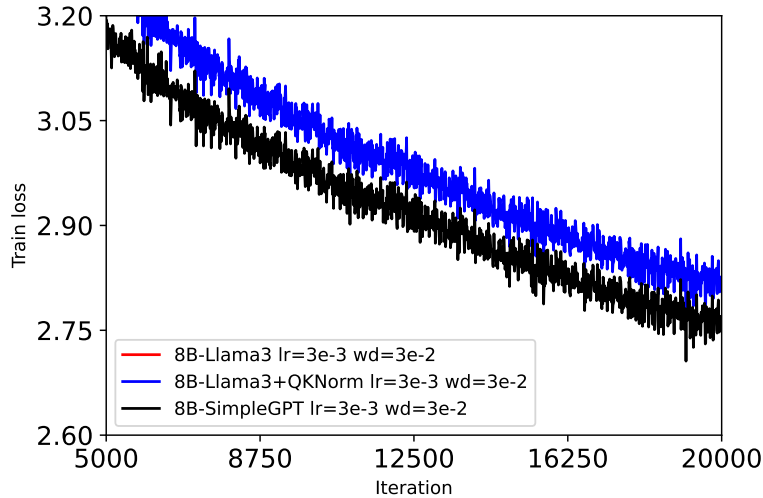


8B-Llama3 lr= $3e-4$ wd= $1e-1$: last = 2.779

8B-Llama3+QKNorm lr= $3e-4$ wd= $1e-1$: last = 2.763

8B-SimpleGPT lr= $3e-4$ wd= $1e-1$: last = 2.709

Figure 13. The training loss curves of Llama3-8B, Llama3-8B with QKNorm and SimpleGPT-8B with learning rate $3e-4$ and weight decay 0.1.



8B-Llama3 lr= $3e-3$ wd= $3e-2$: last = 5.255

8B-Llama3+QKNorm lr= $3e-3$ wd= $3e-2$: last = 2.800

8B-SimpleGPT lr= $3e-3$ wd= $3e-2$: last = 2.738

Figure 14. The training loss curves of Llama3-8B, Llama3-8B with QKNorm and SimpleGPT-8B with learning rate $3e-3$ and weight decay 0.03.

L. More experiments on weight decays

We conduct experiments on the SimpleGPT-8B model using two different weight decay values to evaluate robustness to regularization. Across all tested settings, SimpleGPT-8B consistently outperforms Llama2-8B with QKNorm. These results indicate that the benefits of SimpleNorm are not sensitive to the choice of weight decay, further demonstrating its robustness in large-scale training. Results are shown in Figure 15.

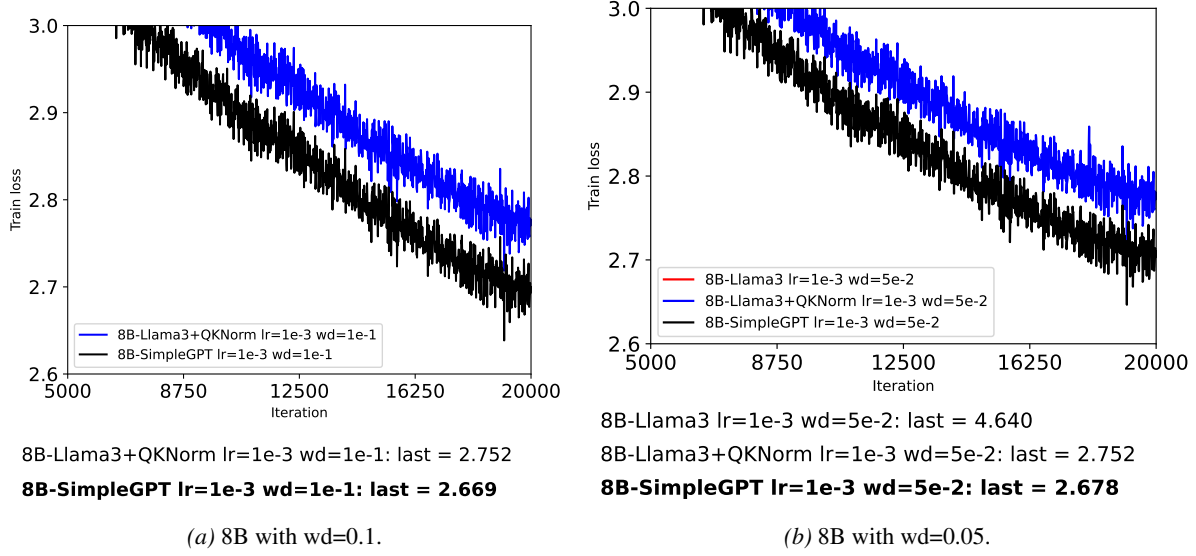


Figure 15. Overall comparison across Llama3-8B with QKNorm and SimpleGPT-1B under two different weight decay values.